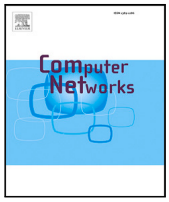
Computer Networks 239 (2024) 110135



Contents lists available at ScienceDirect

Computer Networks

journal homepage: www.elsevier.com/locate/comnet





Graph neural network based robust anomaly detection at service level in SDN driven microservice system

Hongyang Chen [a], Pengfei Chen [a,*], Benran Wang [a], Xian Yu [b,c], Xiaofan Chen [c], Dandan Ma [d], Zibin Zheng [e]

[a] School of Computer Science And Engineering, Sun Yat-sen University, Guangzhou, 510006, Guangdong, China
[b] Shenzhen Institutes of Advanced Technology, Chinese Academy of Sciences, Shenzhen, 511464, Guangdong, China
[c] Sangfor Technologies Inc, Shenzhen, 511464, Guangdong, China
[d] Guangdong Unicom, Guangzhou, 510006, Guangdong, China
[e] School of Software Engineering, Sun Yat-sen University, Zhuhai, 519000, Guangdong, China

ARTICLE INFO

Dataset link: Application Metrics monitoring by SDN based monitor system (Original data)

Keywords:
Software defined network
Microservice
Overlay network
Anomaly detection
Deep learning

ABSTRACT

Software-Defined Network (SDN) has been widely employed in data center networks to host various software systems such as microservice systems. A microservice system is constructed by a microservice architecture composed of dozens of services running in different lightweight virtual machines (e.g., containers) and relying on overlay networks for connectivity. However, with the increase of the system scale and the network traffic volume, the network becomes more complex and common to behave anomalously. Therefore, it is error-prone to manually investigate anomalies. To tackle this problem, we propose a non-intrusive monitoring system to help network operators obtain deep insights from the network traffic generated by the running system. Based on the monitoring system, we propose an anomaly detection method to automatically detect network anomalies. The core idea is to learn behavior patterns of inter/intra-components by modeling spatial–temporal relationships amongst collected network metrics with the proposed model STCell-VAE. The pattern will be leveraged to reconstruct the input data. Subsequently, the reconstruction probability is used to determine anomalies. Additionally, STCell-VAE can assimilate network information from new components as the network evolves, eliminating the necessity for retraining. Experiments are conducted within a real networking system and four simulated large-scale networks. The results demonstrate that our system can detect anomalies with about 0.7 F1-score, which outperforms state-of-the-art methods.

1. Introduction

As cloud technology continues to advance rapidly, an increasing number of modern enterprises have adopted microservice architecture for the development of their software systems and applications. Industrial microservice applications are always large-scale distributed systems, comprising dozens or even thousands of services [1]. These services run on various Virtual Machines (VMs) or lightweight containers, such as Docker [2], which are easily managed using orchestration tools like Kubernetes [3] and Docker Swarm [4].

Unlike traditional monolithic applications, microservice application components need to interact with each other through message passing [5] to serve different user requests. Overlay networks, such as Flannel [6] and Calico [7], are widely adopted in orchestration tools to facilitate connectivity within microservice systems. In an overlay network, each application component is assigned with a private IP address that is independent of the host network. Via tunneling protocols, packets sent from each component are encapsulated inside packets with public (host) IP to support inter-host transmission. The overlay network traffic between application components is called application-specific flows.

Powered by the network agility and flexibility, SDN (Software Defined Networking) has gained widespread adoption in data center networks, and hosts different software systems and mobile applications, including microservice systems. SDN decouples network control (control plane) from forwarding functions (data plane). This decomposition grants SDN the capability of network programmability for network control, allowing packets to be processed using customized programs.

While these technologies simplify development and deployment, they also lead to an explosion in the volume of network flows, causing increased complexity within the network. This complexity poses challenges for network operators in comprehending the performance and availability of the network and applications. In details, some network

* Corresponding author.
E-mail address: chenpf7@mail.sysu.edu.cn (P. Chen).

https://doi.org/10.1016/j.comnet.2023.110135
Received 6 April 2023; Received in revised form 21 September 2023; Accepted 3 December 2023
Available online 6 December 2023
1389-1286/© 2023 Elsevier B.V. All rights reserved.

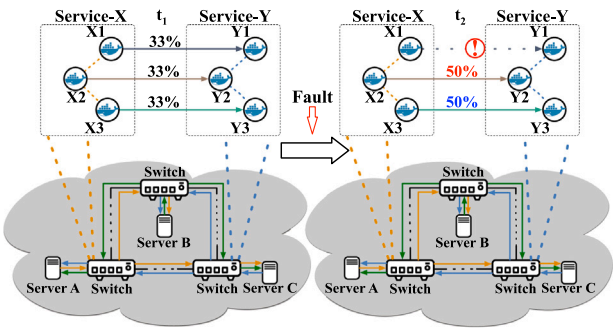


Fig. 1. An example of the hidden network fault. Network flows in physical network are in the lower part and overlay network flows produced from distributed application are in the upper part. The application is built with two services, each of which has three components. The physical network flow from server A to server C comprises three application flows with equal traffic in the normal circumstance (t_1). At time t_2, a network fault between X1 and Y1 occurs. But due to load balance, additional requests are dispatched to other flows (i.e., X2-Y2 and X3-Y3). From the view of physical network, it is hard to detect such a change because the physical network flow keeps the same all the time.

faults occurring within applications are hidden beneath the surface, as exemplified by the fault in Fig. 1. These rarely detected faults result in wasted network bandwidth. Furthermore, the cumulative impact of these faults may ultimately lead to system crashes. Thus, the detection of these hidden faults is of paramount importance. However, achieving this task is non-trivial due to the following challenges.

- **Lack of a fine-grained monitoring system to detect hidden faults.** Traditional network monitoring systems [8–13] always rely on network quintuple (i.e., *source ip, source port, destination ip, destination port, protocol*) to determine which flows or packets to track. With the use of virtualization technology and tunneling protocol, multiple application-specific flows are mixed in quintuple-defined flows. Therefore, these systems cannot monitor application-specific flows hidden behind quintuple-defined flows.
- **Lack of expert knowledge to understand the running application components over the network.** As the network consistently serves diverse microservice applications [1,14–16] which are developed by different software teams, network operators may not have a precise understanding of the intricate details of how the individual components behave and interact with each other. Consequently, evaluating the network status and determining whether faults occur in the network or the running applications [17,18] becomes challenging for them.
- **Lack of approaches that detect anomalies with spatial–temporal information.** Existing approaches for anomaly detection can be broadly categorized into those that utilize temporal information and those that use spatial information. Some temporal-based methods detect anomalies on each component individually [19] or on each metric of the entire system separately [20–22]. Others treat the system as a whole and model only the temporal behavior of the system to detect anomalies [23–27]. On the other hand, spatial-based methods regard each component as a node with multiple attributes and use edges to represent the connectivity between components. Then they [28–34] examine whether there are disruptions in the spatial dependencies between components. Very few of them could evaluate the network state comprehensively.
- **The network behavior changes dynamically.** As the workload imposed on these systems continually evolves over time [35,36], they must dynamically scale out or in [16] to meet the Service Level Agreement (SLA) requirements. Consequently, the number of running components and their network topology inevitably undergo changes. This dynamism poses a challenge to the effectiveness of existing anomaly detection methods [19,23–25], which requires adaption when new components appear.

In this study, we propose a fine-grained network monitoring system leveraging the programmability of SDN. Its primary objective is to monitor the status of application-specific flows, thereby empowering network operators to acquire comprehensive insights into the entire network and uncover hidden network faults. Since the measured network data contains periodic noises that hinder the further detection, we propose a method to eliminate these noises. Then, we propose a deep learning model, STCell-VAE, to perform anomaly detection on the denoised data. During training, STCell-VAE learns the behavior patterns of inter/intra-components by modeling the spatial–temporal relationships amongst collected network data. As normal data significantly outweighs abnormal data, and anomalies are always unpredictable [37–40], the learned behavior pattern in STCell-VAE is more likely to represent the normal behavior pattern. Subsequently, during detection, we utilize the trained model to reconstruct the input data and calculate the reconstruction probability. The reconstruction probability, which reflects deviations of the input data pattern from the learned patterns, is used to determine anomalies. Moreover, given the common occurrence of dynamic changes in network topology, STCell-VAE is designed to integrate the network information of new components, rendering it to be updated without retraining.

Experiments are conducted within a real networking system, where a microservice application is deployed and a production-based workload is generated. Results demonstrate that our system can detect anomalies with an F1-score of approximately 0.7, surpassing state-of-the-art methods. The evaluation also shows that our method can detect anomalies even in dynamic network environments where the running components deployed on the network changes. We also evaluate the scalability of the model within four simulated large-scale networks constructed by Mininet [41].

Contributions of this paper are summarized as follows.

- We propose a novel network monitoring system that provides extra visibility to the network status of application-specific flows in SDN. It helps obtain deep insights into the network and efficiently detect hidden network faults.
- We introduce a robust end-to-end approach for anomaly detection, employing the STCell-VAE model to extract spatial–temporal relationships within monitoring data. This is accomplished through the integration of Graph Convolutional Networks (GCN) and Long Short-Term Memory (LSTM) models. Our method is intentionally designed to withstand periodic noise and remains adaptable in dynamic environments where the number of components changes dynamically.
- We design and implement a real-world prototype to automatically monitor the network and detect anomalies. Our experiments are based on both a simulated large-scale network and a physical SDN testbed hosting a cloud-native benchmark. The experimental results demonstrate the effectiveness of our approach.

The rest of the paper is structured as follows. Section 2 presents the previous related work. Section 3 presents our motivation and formulates the problem. Section 4 introduces the overview of the proposed network monitoring system and depicts its design. Section 5 outlines and specifies the proposed anomaly detection approach. Section 6 shows the experimental evaluation and Section 7 concludes this paper.

2. Related work

2.1. Network monitoring systems

Traditional network monitoring: Traditional network monitoring systems that are designed to monitor the physical network can be divided into two categories, namely flow-level measurement and packet-level measurement. Existing flow-level measurement studies [11–13,

42,43] measure the network per-flow, rendering the analysis for flow-level data to detect anomalies in the network. However, these studies only analyze flow-level data and they are unable to detect packet-level anomalies. Therefore, another line of research targets at collecting packet-level information [8–10,33,34,44–49] and visualizing the network in a finer granularity. These methods either use packet mirroring at switches to monitor the network [8–10,45,49], or send probing packets into the network to estimate the network status [33,34,46–48]. Specifically, Planck [9], OpenSAFE [45] and NetSight [8] leverage switch mirroring to monitor the network and debug network traces. Although these methods show the ability to capture traces of packets and concisely retrieve packet histories, their overhead is high because they need to transmit and process all traffic. To lower overhead, Everflow [10] uses "match" to select the right traffic to mirror, which also inspires the design of our method. Other methods measures the states of devices or links by sending probing packets [33,34,46–48], which would be collected by a processor to detect and diagnose the network failures. However, the probe traffic and the production traffic may travel along various paths, hindering accurate and timely diagnosis.

Virtualized network monitoring: Traditional network monitoring methods are designed to measure the physical network and cannot be directly applied to monitor the network traffic generated within virtualized networks on the physical network. To address this, several existing approaches have been proposed to monitor the virtualized network [28,29,50–57]. These methods either collect network flow statistics [29,52,54–57] or gather packet-level information [28,50,51,53]. Specifically, V-Sight [29], OFRwind [54], VeriFlow [52], SketchVisor [55] and LightGuardian [56] targets at capturing accurate flow-level statistics on each participating network device. However, the granularity of monitoring is constrained due to encapsulation of flows passing through the network devices. On the other hand, other methods target at obtaining packet-level information. For example, VNet-Pingmesh [28] utilizes ping tool to test the network connection status between different virtual forward devices. Since VNet-Pingmesh needs to send probing packet to test the network status, hindering to find failures timely, NetAlytics [51] leverages SDN, NFV to gather and process packet-level data, and then utilizes these data to analyze the distributed application performance. However, the adoption of NetAlytics requires the resource consumption on host servers. Drawing insights from these existing works, our network monitoring system concentrates on gathering packet-level information of traffic produced by distributed applications deployed in the virtualized overlay network, while avoiding resource consumption on host servers.

2.2. Unsupervised anomaly detection on network

Unsupervised anomaly detection methods have been developed and widely used in the past years. These methods can be categorized into traditional methods and deep learning models.

Traditional methods can be divided into three categories, namely statistical approaches such as Gaussian-based model, classification based approaches such as One-Class Support Vector Machines (OC-SVM [58]) and proximity-based approaches such as KNN [59]. The performance of these traditional methods usually relies on the assumption of normal data. Their performance diminishes when the data fails to conform to these corresponding assumptions.

Since deep learning models work well on modeling complex relations in data and always perform better, they become more and more popular. LSTM-AD [23] is a prediction based model, which uses stacked LSTM networks to train a prediction model. Then the prediction error is used as the measure of anomaly. Donut [19], LSTM-ED [24], and OmniAnomaly [25] belong to reconstruction-based models. Specifically, Donut and OmniAnomaly leverages VAE [60] to reconstruct time series data and use the reconstruction error as the measure of anomaly. However, Donut [19] reconstructs univariate time series while OmniAnomaly [25] uses LSTM to extract the temporal dependency in time series data and combine it with VAE to reconstruct multivariate time series. Same as OmniAnomaly, LSTM-ED [24] adopts an LSTM-based seq2seq auto-encoder to reconstruct multivariate time series and also uses reconstruction error to detect anomalies. RANSynCoders [27] proposes an architecture that employs multiple encoders and decoders to infer and localize anomalies through majority voting, and uses decoders to determine the bounds of the reconstructions. With the development of adversarial technique, USAD [26] combines the reconstruction based learning and adversarial training by adopting an autoencoders architecture within an adversarial training framework. The models employed for anomaly detection rely on modeling temporal relationships within time series data. However, they tend to overlook spatial dependencies among individual components, thus limiting their capacity to comprehensively identify faults. Furthermore, while these models perform well under stable conditions with a consistent number of metrics and sufficient historical data for training, there is a need for an adaptive approach that can operate effectively in dynamic environments where the number of components and metrics changes without the necessity for retraining.

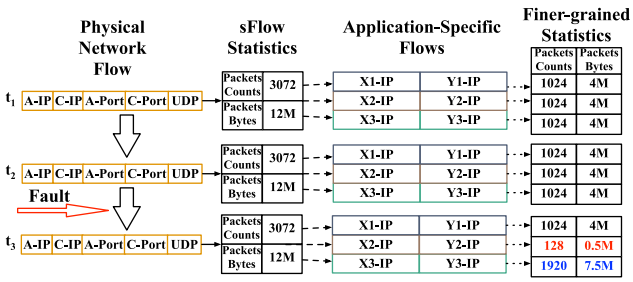


Fig. 2. A fault case that cannot be detected with sFlow.

3. Motivation and problem formulation

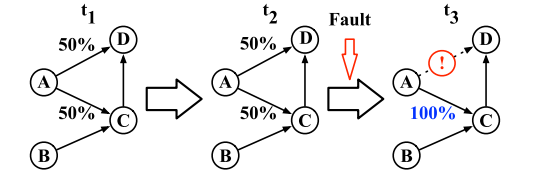
3.1. Motivation

There are two key ideas in this paper, namely *monitoring the application-specific traffic decoupled from quintuple-defined traffic* and *combining spatial–temporal information in modeling*.

sFlow [61] is one of the popular flow-level measurement tools. Network operators leverage it to measure flows in the physical network (quintuple-defined flows), such as monitoring three flows between hosts A, B, C in the lower part in Fig. 1. However, in the context of virtualized distributed applications, flows traverse between physical hosts and encompass multiple application-specific flows. Network status changes of application flows interfere with each other. Relying solely on the tracking of quintuple-defined flows may offer limited insights into this interference, potentially missing certain hidden faults.

For example, in Fig. 2, the physical flow *A to C* aggregates three application-specific flows whose statistics are the same in normal circumstances (e.g. t_1, t_2). At time t_3, a fault leads to a dramatic change in statistics of two application-specific flows (e.g.*X2 to Y2, X3 to Y3*), but it does not manifest in the physical flow whose statistics remain unchanged. Thus, we propose application-specific traffic monitoring beyond quintuple-defined traffic, which gives network operators finer-grained insights into the network and helps them identify non-trivial faults.

On the other hand, most existing network diagnosis systems detect and diagnose faults from an incomplete view. Specifically, some temporal-based methods detect anomalies by analyzing metrics from each component individually, which provides component-level information [19]. Other approaches consider the system as a whole and either detect anomalies on each metric of the entire system separately [20–22] or model the temporal behavior of the whole system

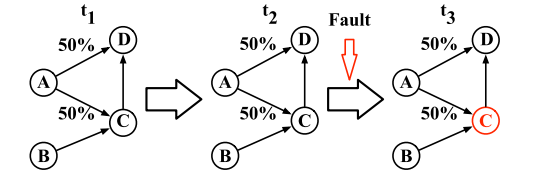


Node	Sent	Recv
A	10	0
B	1000	0
C	1005	1005
D	0	1005

Node	Sent	Recv
A	10	0
B	1000	0
C	1005	1005
D	0	1005

Node	Sent	Recv
A	10	0
B	1000	0
C	1010	1010
D	0	1010

(a) **The case that cannot be detected with temporal methods**



Node	Sent	Recv
A	10	0
B	1000	0
C	1005	1005
D	0	1005

Node	Sent	Recv
A	10	0
B	1000	0
C	1005	1005
D	0	1005

Node	Sent	Recv
A	10	0
B	1000	0
C	10	1005
D	0	10

(b) **The case that cannot be detected with spatial methods**

Fig. 3. Two fault cases that cannot be detected with only temporal or spatial based methods. Metrics of each component include *sent packets counts* and *received packets counts*.

and detect anomalies [23–27]. In contrary, spatial-based methods [28–34] focus on the interaction of components. They either detect whether a connectivity problem exists or whether requests between network devices fail. Fig. 3 shows two cases that cannot be detected by only considering partial information. As we can see, the statistics of each component remain unchanged in normal circumstances (e.g. t_1, t_2). The network statistics of components B, C, D are in the similar range, while they are quite greater than the value of A. In Fig. 3(a), a connection failure between A and D makes all packets from A send to C. Therefore, statistics of C and D slightly change at t_3 (from 1005 to 1010). However, the changes are too subtle to be detected if using temporal-based methods. In Fig. 3(b), a node failure of C results in a significant decrease of packets sent from C. Although statistics of C and D change a lot (from 1005 to 10), the failure cannot be detected by spatial-based methods because connections between components are healthy. The incompleteness resulting from the unilateral consideration inspires us to include topological information to represent a network system state and combine topological information with the temporal-based method.

3.2. Problem formulation

Table 1 summarizes all notations used in the model. This study focuses on multivariate time series, denoted as $\mathbf{X} = \{\mathbf{X_1}, \mathbf{X_2}, \ldots, \mathbf{X_T}\}$ and adjacency matrices $\mathbf{A} = \{\mathbf{A_1}, \mathbf{A_2}, \ldots, \mathbf{A_T}\}$ which represents the application topology. An observation in $\mathbf{X}$, denoted as $\mathbf{X_t} = \{x_{t,1}, x_{t,2}, \ldots, x_{t,N}\}$, contains multivariate metrics of N components at time t. Then, we use $\mathbf{X}_{t-W+1:t}$ to denote a sequence of observations $\{\mathbf{X_{t-W+1}}, \ldots, \mathbf{X_t}\}$ from $t-W+1$ to t, while the adjacency matrices are denoted as $\mathbf{A_{t-W+1:t}}$.

As [62] presents, a multivariate time series anomaly is a data point or a data sequence that deviates from the normal behavior. In practice, an anomaly indicates the system goes wrong which needs further investigation for failure recovery. Formally, graph based multivariate anomaly detection is defined as followed. For time t, given a sequence of historical observations $\mathbf{X}_{t-W+1:t}$ as well as adjacency matrices $\mathbf{A_{t-W+1:t}}$, we determine whether an anomaly occurs. We address this problem by modeling spatial dependency using the topological information with GCN, followed by an LSTM to capture long-term dependencies in the time series data. As is known, Convolutional Neural Networks (CNN) has been widely adopted on a wide range of machine learning problems [63,64]. They are proficient at extracting information from graphs in Euclidean domains [65]. However, the topology of microservice systems are always random whose nodes have an arbitrary number of neighbors [66]. Consequently, the corresponding graph of a microservice system operates in non-Euclidean domains, hindering the adoption of CNN. Additionally, the attention has also been explored to enhance graph embedding learning [67–69]. However, all these methods require a multi-step pipeline including random walk generation and semi-supervised training, wherein each step has to be optimized separately [70]. Therefore, GCN is selected. We also use VAE to reconstruct time series data, and the difference between original time series data and reconstructed data is utilized to detect anomalies.

Table 1
Notations and descriptions.

Notations	Descriptions
N	The number of components in the distributed application over SDN, indexed by i.
F	The number of metrics collected in each component, indexed by j.
S_h	The dimensions of the hidden state.
T	The number of observations.
$x_{t,i,j}$	The ith component's jth metrics at time t.
W	The length of the sliding window.
$\mathbf{X} \in \mathbf{R}^{T\times N\times F}$	A multivariate time series recording metrics of components in all observations.
$\mathbf{X}_t \in \mathbf{R}^{N\times F}$	The observation of $\mathbf{X}$ at time t.
$\tilde{\mathbf{X}} \in \mathbf{R}^{T\times N\times F}$	The pre-processed data of $\mathbf{X}$.
$\mathbf{A} \in \mathbf{R}^{T\times N\times N}$	The adjacency matrices of the application topology in all observations.
$\mathbf{A_t} \in \mathbf{R}^{N\times N}$	The adjacency matrices of the application topology at time t.

4. System design

4.1. System overview

The overall framework of our approach is shown in Fig. 4. It comprises two main components, namely the network monitoring system and the anomaly detection system.

The network monitoring system consists of four key modules, *Coordinator* (marked as ①), *Data Collector* (marked as ②), *Statistics Aggregator* (marked as ③) and *Storage* (marked as ④). *Coordinator* periodically distributes pre-defined "match and mirror" rules to the SDN controller and activates a group of *Data Collectors*. The SDN controller installs rules to network forward devices (e.g. switches), and packets that match these rules will be mirrored to *Data Collectors*. Then *Data Collector* extracts original packets from the payload of packets and get packet-level statistics. After that, *Statistics Aggregator* retrieves the statistics from distributed *Data Collectors*, constructs the final statistics, builds the communication topology, and stores them into *Storage*. Finally, the anomaly detection system uses the collected data to detect anomalies which will be introduced in detail in Section 5.

4.2. Coordinator

Coordinator determines which "match-and-mirror" rules should be adopted. We have noticed that the most popular tunneling protocol to create an overlay network in microservice systems is Virtual Extensible Local Area Network (VxLAN). VxLAN encapsulates packets sent from

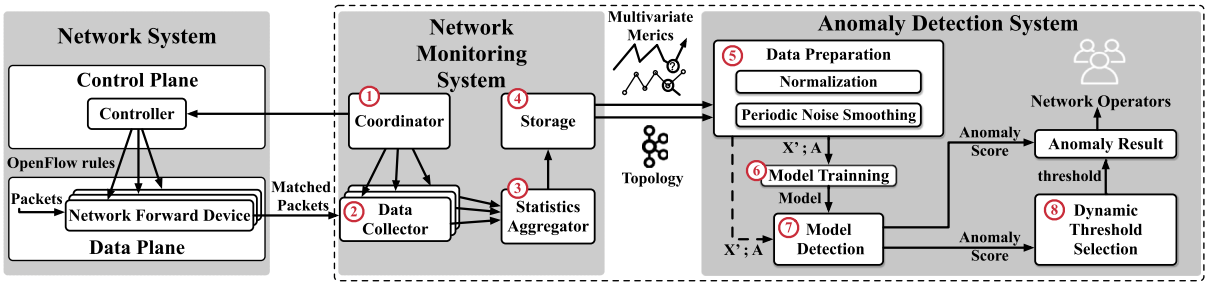


Fig. 4. The framework of our system.

application components into UDP packets and transmits them over the IP network. It uses a fixed UDP port (i.e., 8472) for sending encapsulated packets. Therefore, the type of monitored flows can be identified by specifying flows destined for or originated from a particular UDP port. The values of protocol type (UDP) and port number (8472) are translated into the *match* portion of an OpenFlow rule. Furthermore, packets pass through multiple switches in the data center network. If the same packet is mirrored to *Data Collector* by multiple switches and counted several times, the final monitoring results will not be correct. Therefore, we design a rule to render each switch to only mirror those packets whose first hop in the network is it. We implement the rule by restricting the IP addresses of the mirrored packets to the IP of the host devices that are directly connected to the switch. The rule is also translated into the *match* portion.

Each *Data Collector* is deployed in a single switch and only monitors target flows as stated in Section 4.3. Therefore, the *action* portion of the OpenFlow rule is defined with a standard output port sending to the destination and a secondary output port forwarding to *Data Collector*. *Coordinator* integrates match and action criteria to define OpenFlow rules and periodically distributes them to the SDN controller. Then, the controller proactively pushes rules to switches, permitting switches to mirror a copy of matched packets to *Data Collectors* for further analysis.

Sending all packets generated from applications bring significant overhead if the workload of applications is heavy. To balance the monitoring overload and the detection accuracy, the sampling rate should be determined carefully. Therefore, an API of *Coordinator* is provided to network operators which can be used to tune the sampling rate to adapt to different scenarios.

4.3. Data collector

To monitor network flows through different switches, a group of *Data Collectors* are deployed. One *Data Collector* only monitors one switch. When packets arrive, *Data Collector* de-capsulates them, extracts the payload and retrieves original packets produced by application components. Original packets are used to extract the information about application-specific flows between various application components. These flows are characterized by virtualized source component IP and virtualized destination component IP.

Then, *Data Collectors* accumulate the number of packets and bytes generated by each application-specific flow. The accumulation statistics are periodically sent to *Statistics Aggregator* for further aggregation.

Since each *Data Collector* simply extracts a small amount of data from matched packets and holds statistics for each flow, its overhead is reduced significantly. Furthermore, each *Data Collector* only tracks flows over one switch. Finally, unlike other monitoring systems that analyze entire raw packets or packet headers, *Data Collector* further reduces the overhead by aggregating packets and then sending the statistics to *Statistics Aggregator*.

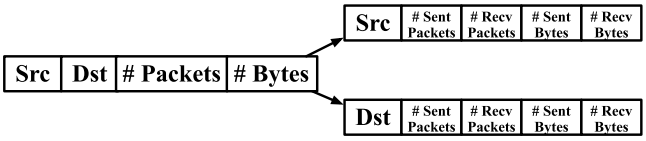


Fig. 5. The transformation from flow statistics to components statistics.

4.4. Statistics aggregator

Statistics Aggregator obtains network statistics for each flow from each *Data Collector*, and computes statistics for each component by aggregating flows that belong to the same source component or the same destination component. Statistics per component include *the number of sent packets*, *the sent byte counts*, *the number of received packets*, and *the received byte counts*. Fig. 5 shows the transformation from flow statistics to components statistics. *Statistics Aggregator* manages the map from each component to the statistics. When the statistics for a flow comes, *Statistics Aggregator* would add the related statistics to *the number of sent packets* and *the sent byte counts* of the source component and add the related statistics to *the number of received packets*, and *the received byte counts* of the destination component.

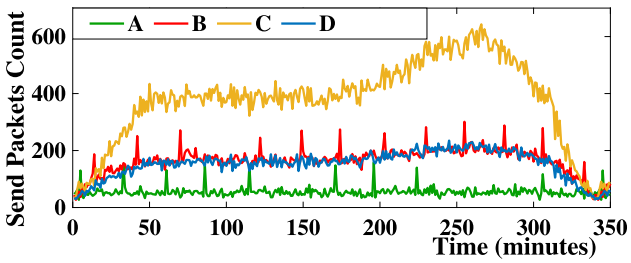
Statistics Aggregator also builds the communication topology which will be utilized as the input of the proposed graph based model. The node of the topology represents the components that have been seen and the edge of the topology represents the observed connection between components. In detail, when the network statistics of each flow are received, *Statistics Aggregator* will add a new node to the topology if the source component or the destination component is not recorded before. Moreover, *Statistics Aggregator* will add a new edge to the topology if the connection between the source component and the destination component is not observed before. The weight of the edge is set to 1. Finally, the statistics per component and the topology are periodically transmitted to *Storage* for saving.

5. Graph based model for anomaly detection

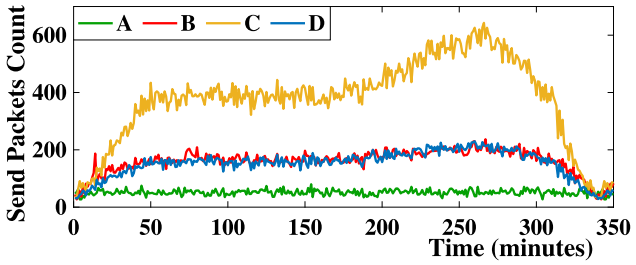
As presented in Fig. 4, the anomaly detection system consists of four modules, namely Data Preparation (marked as ⑤), Model Training (marked as ⑥), Model Detection (marked as ⑦) and Dynamic Threshold Selection (marked as ⑧). Multivariate time series generated from statistics per component at each observation are fed into the anomaly detection system along with the communication topology information. In this section, we will introduce these modules in detail.

5.1. Data preparation

Data Preparation module is shared by Model Training and Model Detection. Through preparation, time series metrics from different components are normalized first. Then Periodic Noise Smoothing is used for the normalized time series metrics to eliminate the negative impact of periodic local fluctuations.



(a) **Before PNS**



(b) **After PNS**

Fig. 6. Multiple time series in different components (A, B, C, D) before and after Periodic Noises Smoothing (PNS). A and B have periodic noises (i.e. fluctuations), while C and D do not have. Other metrics of other components follow the same pattern.

5.1.1. Normalization

Scales of different metrics are usually different. Deep learning models always perform better on the normalized data. Therefore, time series metrics are normalized. In detail, the jth metrics of all the components in the training data are concatenated to calculate the mean (μ_j^{train}) and the standard deviation (σ_j^{train}). Then, the jth metrics of the ith node at time t, $x_{t,i,j}$ is normalized using μ_j^{train} and σ_j^{train} with the calculation as Eq. (1).

$$\tilde{x}_{t,i,j} = \frac{x_{t,i,j} - \mu_j^{train}}{\sigma_j^{train}} \tag{1}$$

5.1.2. Periodic noise smoothing

Heartbeat is a periodic signal generated by hardware or software to indicate normal operation or to synchronize other parts of a computer system [71]. Distributed applications utilize such mechanism to sense the runtime status of the application components. For example, the database will send heartbeat packets to its clients to indicate whether their connections are healthy or not. In this study, we notice that the heartbeat packets transferred between application components cause some relatively large local fluctuations in the network status of application components periodically from our observation. However, reconstruction-based anomaly detection models are sensitive to these large noises.

In Fig. 6(a), there are some large local fluctuations in different time series which occur periodically. Although the occurrence of fluctuations does not represent anomalies, they result in false alarms of anomaly detection methods. Therefore, a simple but efficient method, Spectral Residual (SR) [72,73], is utilized to smooth these fluctuations. When a new time series comes, SR calculates the anomaly score of each observation.

Specifically, SR contains three major steps: **(i)** Fourier Transform to get the log amplitude spectrum, **ii)** Calculation of *spectral residual*; and **iii)** Inverse Fourier Transform that transforms the sequence back to spatial domain. Given the jth univariate metrics sequence of the ith component, $\mathbf{x}$, the saliency map $S(\mathbf{x})$ can be calculated as:

$$A(f) = Amplitude(\mathcal{F}(\mathbf{x})) \tag{2}$$

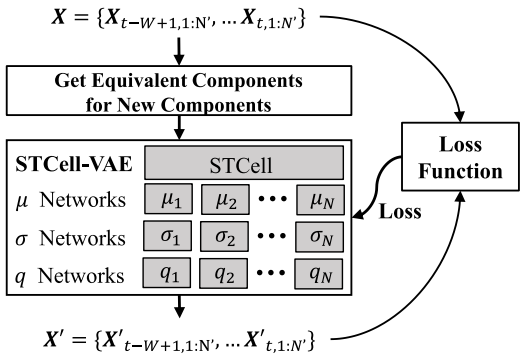


Fig. 7. The overview of the detection model.

$$P(f) = Phrase(\mathcal{F}(\mathbf{x})) \tag{3}$$

$$L(f) = log(A(f)) \tag{4}$$

$$AL(f) = h_q(f) \cdot L(f) \tag{5}$$

$$R(f) = L(f) - AL(f) \tag{6}$$

$$S(\mathbf{x}) = |\mathcal{F}^{-1}(exp(R(f) + iP(f)))| \tag{7}$$

In these equations, $\mathcal{F}$ and $\mathcal{F}^{-1}$ represent the Fourier Transform and Inverse Fourier Transform, respectively. $A(f)$ and $P(f)$ stand for the amplitude spectrum and the corresponding phase spectrum of sequence $\mathbf{x}$. $L(f)$ denotes the log representation of $A(f)$, and $AL(f)$ is the average spectrum of $L(f)$, which is approximated by convoluting the input sequence with $h_q(f)$. $h_q(f)$ is an $q \times q$ matrix defined by

$$h_q(f) = \frac{1}{q^2} \times \begin{bmatrix} 1 & 1 & \ldots & 1 \\ 1 & 1 & \ldots & 1 \\ \vdots & \vdots & \ddots & \vdots \\ 1 & 1 & \ldots & 1 \end{bmatrix} \tag{8}$$

$R(f)$ represents the *spectral residual*, which is calculated by subtracting the average log spectrum $AL(f)$ from the log spectrum $L(f)$. Subsequently, the *spectral residual*, serving as a compressed representation of the sequence, is transformed back to the spatial domain via Inverse Fourier Transform. The resulting sequence $S(\mathbf{x})$, referred to as the *saliency map*, is considered as the anomaly score for each time point of sequence $\mathbf{x}$. If the anomaly score exceeds the threshold (set as 3 by default), the corresponding time point is labeled as an anomaly.

Then, a simple rule is adopted to determine whether the labeled time points should be smoothed. Suppose the labeled time point is P_t and the period of fluctuations is l (set as 20 in this study). For time points of the previous pth periods before P_t, we check whether they are labeled by SR. If yes, P_t is regarded as the time point that should be smoothed and the value of P_t is replaced with a neighbor value. The result of periodic smoothing is presented in Fig. 6(b).

5.2. Model training

5.2.1. Overall architecture

We first begin by providing an overview of our proposed method, as depicted in Fig. 7. The key of the proposed method is a neural network named STCell-VAE which is presented in Fig. 9. The STCell-VAE comprises two distinct neural networks: the STCell network, which delineates the overarching relationships between components (as depicted in Fig. 8), and three dense networks (μ, σ, q) which model the specific characteristics of each individual component. In details, STCell, serving as the fundamental building block of STCell-VAE, captures the comprehensive spatial–temporal relationship between each component within the input data at each observation. The μ networks and σ networks

derive the means and standard deviations of the posterior distribution of metrics associated with each independent component. The q dense networks approximate the distributions $q(x_{t,i}|z_{t,i})$ for metrics of each component (i.e., i). Finally, the reconstruction of $\mathbf{X}$ is accomplished through sampling from these modeled distributions. The model takes historical observations data $\mathbf{X}$ as input. This data encompasses both multivariate time series data and the adjacency matrix. The time series data from N components is denoised, and the data preparation process has been described in Section 5.1. Once the model is obtained, the anomaly score of a new observation can be calculated with it. During the detection stage, a dynamic threshold selection strategy is adopted to choose a threshold. If the anomaly score of an observation exceeds the threshold, the observation will be considered abnormal. The details of anomaly score calculation and threshold selection are presented in Section 5.3.

Note that, N components contains N' initial components and $N-N'$ components that are created unpredictably during training or detecting. Before these data are fed into STCell-VAE, N components are divided into different equal classes based on comparisons of their neighbors. Subsequently, the original component within the same class that exhibits a closer temporal similarity in metrics with the new component is identified as its equivalent component. The new component shares the same dense networks as their equivalent original component in training and detection. Consequently, STCell-VAE can be updated and the network information is incorporated as the network evolves.

5.2.2. Calculation of equal class and equivalent components

As the systems deployed over the network always scale dynamically to meet the SLAs and new services are regularly introduced to fulfill evolving customer requirements, the topology of systems always changes. Therefore, new components may emerge unexpectedly during model training or even during detection, whose behavior has never been learned by the model before. Consequently, time series metrics of these components cannot be reconstructed without retraining the model. To address this issue, we have devised a method for identifying an existing component within the system that exhibits similar behavior to the new component. Subsequently, we utilize the trained model of the identified original component to reconstruct the metrics of the new component.

$$D_{i,j} = \sqrt{(x_{t:t+W',i} - x_{t:t+W',j})^2} \tag{9}$$

The fundamental insight behind the method for identifying an equivalent component for the new component is based on the expectation that two equivalent components should exhibit similarity in terms of their neighbors connected via network and the patterns of their metrics. Therefore, when a new component emerges at time t, our method first groups all components into distinct equal classes by comparing their neighbors. The components of the same equal class should have the same neighbor components. Then, after a time interval (W'), we calculate the Euclidean Distance between the metrics ($x_{t:t+W',i}$) of new component i and metrics ($x_{t:t+W',j}$) of other original component j within the same equal class following Eq. (9). The original component whose time series metrics have the greatest proximity to those of the new component in Euclidean Distance is designated as the equivalent component. Specifically, if a new component does not share completely the same neighbor components as other original components, we select three original components that have more similar neighbors as the new components to constitute the equal class of the new component.

5.2.3. STCell

To simultaneously capture the spatial–temporal relationship of multivariate time series, STCell is built as a basic block of the model. As shown in Fig. 8, STCell consists of two main components, namely GCN and LSTM. Specifically, GCN constructs components' graph-based representations considering features per component and the topology.

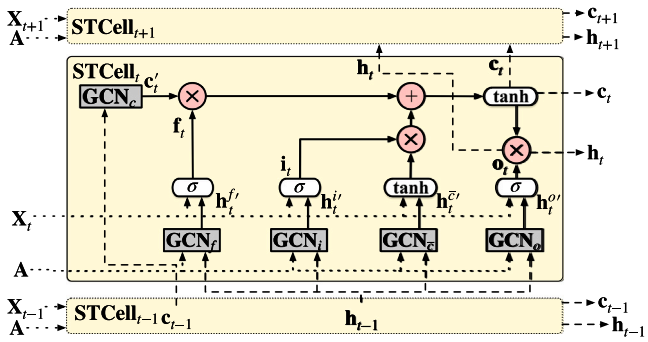


Fig. 8. The architecture of STCell.

LSTM captures the dynamic change of application components' network metrics.

$$\mathbf{h}'_t = \tilde{\mathbf{D}}^{-\frac{1}{2}}\tilde{\mathbf{A}}\tilde{\mathbf{D}}^{-\frac{1}{2}}\mathbf{h}_{t-1}\mathbf{W} \tag{10}$$

Eq. (10) shows the calculation of graph convolution operation on the hidden state $\mathbf{h}_{t-1} \in \mathbf{R}^{N\times S_h}$, using the adjacency matrix $\mathbf{A}$ of the topology as another input. In this equation, $\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}$ denotes the adjacency matrix with self-inserted loops. $\tilde{\mathbf{D}}$ represents its degree matrix and its element is calculated by $\tilde{D}_{ii} = \sum_j \tilde{A}_{ij}$. $\mathbf{W}$ is a trainable weight matrix. The operation on the cell state $\mathbf{c}_{t-1}$ is similar to the hidden state $\mathbf{h}_{t-1}$, except that the input is changed from $\mathbf{h}_{t-1}$ to $\mathbf{c}_{t-1}$.

At time t, STCell uses five GCN models to respectively perform graph convolution operations on results of the last time point, $\mathbf{c}_{t-1}$ and $\mathbf{h}_{t-1}$. By performing convolution operations with various GCN models, $\mathbf{h}_t^{f'}$, $\mathbf{h}_t^{i'}$, $\mathbf{h}_t^{o'}$, $\mathbf{h}_t^{\bar{c}'}$ and $\mathbf{c}'_t$ are obtained, which then be respectively used as the input to different gates of the LSTM. The calculations conducted by the LSTM can be formulated as Eq. (11), in which $\mathbf{W}_f, \mathbf{W}_i, \mathbf{W}_c, \mathbf{W}_o$ are trainable weights, $\mathbf{b}_f, \mathbf{b}_i, \mathbf{b}_c, \mathbf{b}_o$ are bias matrices and $\odot$ is hadamard product.

$$\begin{aligned}
\mathbf{f}_t &= \sigma(\mathbf{W}_f\mathbf{X}_t + \mathbf{h}_t^{f'} + \mathbf{b}_f)\\
\mathbf{i}_t &= \sigma(\mathbf{W}_i\mathbf{X}_t + \mathbf{h}_t^{i'} + \mathbf{b}_i)\\
\bar{\mathbf{c}}_t &= tanh(\mathbf{W}_c\mathbf{X}_t + \mathbf{h}_t^{\bar{c}'} + \mathbf{b}_c)\\
\mathbf{c}_t &= \mathbf{f}_t \odot \mathbf{c}'_t + \mathbf{i}_t \cdot \bar{\mathbf{c}}_t\\
\mathbf{o}_t &= \sigma(\mathbf{W}_o\mathbf{X}_t + \mathbf{h}_t^{o'} + \mathbf{b}_o)\\
\mathbf{h}_t &= \mathbf{o}_t \odot tanh(\mathbf{c}_t)
\end{aligned} \tag{11}$$

5.2.4. STCell-VAE

The neural network architecture of STCell-VAE is shown in Fig. 9, which is composed of an encoder and a decoder. At time t, a historical time series sequence $\mathbf{X}_{t-W+1:t}$ and an adjacency matrix A are input to the encoder. Let t_0 denote the beginning of sliding window $t-W+1$, $\mathbf{X}_{t_0:t}$ denote $\mathbf{X}_{t-W+1:t}$, ϕ denote encoder network parameters and θ denote decoder network parameters. The encoder uses STCell to extract temporal and spatial features by iteratively feeding every time step of $\mathbf{X}_{t_0:t}$ into the encoder. After consuming the entire input sequence, the hidden state $\mathbf{h}_t$ generated by the encoder is fed into two dense networks to estimate the mean $\mu_\phi(\mathbf{X}_{t_0:t})$ and standard deviation $\sigma_\phi(\mathbf{X}_{t_0:t})$ of the posterior $p_\phi(\mathbf{Z}_t|\mathbf{X}_{t_0:t})$. Then, the randomly sampled $\mathbf{z}_t$ from the posterior $p_\phi(\mathbf{Z}_t|\mathbf{X}_{t_0:t})$ is fed into the decoder which uses a similar structure to recover the sequence in reverse order. At each time, the input for the reconstruction comes from the preceding reconstruction. Specifically, at time $t-1$, the input for the reconstruction is the mean of $q_\theta(\mathbf{X}_t|\mathbf{Z}_t)$. The output for the reconstruction at time k is a distribution which can be represented as the mean $\mathbf{X}'_k$ and the co-variance σ'_k. Note that three dense networks (μ, σ, q) contains a collection of individual networks, each dedicated to modeling either the means or standard deviations of posterior distribution $p(Z_{t,i}|X_{t-W+1:t,i})$ for each component (i.e., i), or models the distributions $q(X_{t,i}|Z_{t,i})$. The number of dense networks is

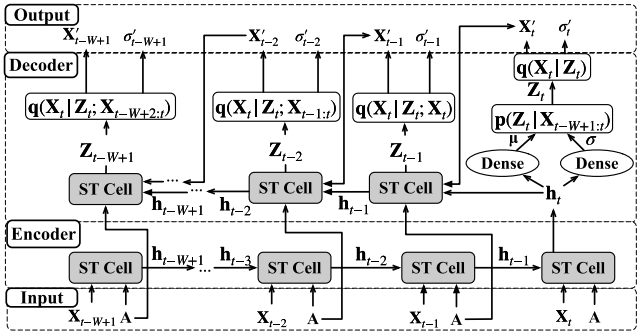


Fig. 9. The overall architecture of STCell-VAE.

determined by the number of original components at the beginning of training. As shown in Fig. 7, the total number of μ (, σ and q) networks is N, where N represents the number of the original components. Therefore, when a new component i emerges, the trained dense networks (μ_j, σ_j, q_j) corresponding to the equivalent original component (i.e., j) are employed to derive the mean and standard deviation of the posterior distribution $p(Z_{t,i}|X_{t-W+1:t,i})$, as well as the distribution $q(X_{t,i}|Z_{t,i})$. The network information of the original component and the new component will be utilized simultaneously to update these networks.

5.2.5. Loss function

The model is trained by optimizing the loss function which is called the evidence lower bound (ELBO) [60]. Given a multivariate time series $\mathbf{X}_{t_0:t}$, the mathematical format is presented as Eq. (12), which is composed of two parts. The first part is the reconstruction probability which measures the probability density of the input in the output's distribution. The second part regularizes the latent variable $\mathbf{Z}_t$ by minimizing the KL divergence between the posterior and the prior of the latent variable.

$$\begin{aligned} L(\mathbf{X}_{t_0:t}) = & E_{p_\phi(\mathbf{Z}_t|\mathbf{X}_{t_0:t})}(log(q_\theta(\mathbf{X}_{t_0:t}|\mathbf{Z}_t))) \\ & - D_{KL}(p_\phi(\mathbf{Z}_t|\mathbf{X}_{t_0:t})||q_\theta(\mathbf{Z}_t)) \end{aligned} \tag{12}$$

In general, a standard normal distribution $N(0, I)$ is used for the prior of $\mathbf{Z}_t$, which simplifies the second term of the Eq. (12). Moreover, the Monte Carlo integration is utilized to calculate the reconstruction probability. Finally, Eq. (12) can be simplified and transformed to Eq. (13), where L denotes the number of samples and $\mathbf{Z}_t^l$ $(l = 1, 2, \ldots, L)$ are sampled from the posterior $p_\phi(\mathbf{Z}_t|\mathbf{X}_{t_0:t})$.

$$\begin{aligned} L(\mathbf{X}_{t_0:t}) = & \frac{1}{L}\sum_{l=1}^{L}\sum_{i=t_0}^{t} log(q_\theta(\mathbf{X}_i|\mathbf{Z}_t^l, \mathbf{X}_{i+1:t})) \\ & - D_{KL}(p_\phi(\mathbf{Z}_t|\mathbf{X}_{t_0:t})||N(0, I)) \end{aligned} \tag{13}$$

5.3. Model detection

5.3.1. Anomaly score

After reconstructing the time series sequence $\mathbf{X}_{t_0:t}$, the reconstruction probability $E_{p_\phi(Z_t|\mathbf{X}_{t_0:t})}(log(q_\theta(\mathbf{X}_t|Z_t)))$ is always used to determine whether an observation at time t is reconstructed precisely. As stated in [19,25], if an observation follows the normal patterns of time series, it can be reconstructed with a high confidence. Otherwise, it is less likely to be reconstructed and the reconstruction probability is small. Therefore, the additive inverse of the reconstruction probability is used as the anomaly score in our model. A higher anomaly score implies a more anomalous observation. The mathematical format of the anomaly score at time t is presented as Eq. (14).

$$\begin{aligned} S_t &= E_{p_\phi(Z_t|\mathbf{X}_{t_0:t})}(log(q_\theta(\mathbf{X}_t|\mathbf{Z}_t))) \\ &= \frac{1}{L}\sum_{l=1}^{L} log(q_\theta(\mathbf{X}_t|\mathbf{Z}_t^l)) \end{aligned} \tag{14}$$

5.3.2. Dynamic threshold selection

After the anomaly score for each observation is obtained, the threshold should be chosen to trigger an alert. If the score of an observation is greater than the threshold, this observation should be classified as an anomaly. In this study, we use a non-parametric, dynamic unsupervised threshold selection approach (NDT) proposed in [74]. Given a set of anomaly scores $\mathbf{s}$ of historical observations, NDT firstly constitutes the set $\mathbf{e}$ using Eq. (15), where the size of $\mathbf{s}$ is S and $\mathbf{z}$ is an ordered set of positive values. As stated in [74], we set values for $\mathbf{z}$ in a range of 2 to 10.

$$\mathbf{e} = \mu(\mathbf{s}) + \mathbf{z}\sigma(\mathbf{s}) \tag{15}$$

Then, the threshold e is determined by

$$e = argmax(\mathbf{e}) = \frac{(\Delta\mu(\mathbf{s})/\mu(\mathbf{s})) + (\Delta\sigma(\mathbf{s})/\sigma(\mathbf{s}))}{|\mathbf{s}_a| + |\mathbf{E}_{seq}^2|} \tag{16}$$

$$\begin{aligned} \Delta\mu(\mathbf{s}) &= \mu(\mathbf{s}) - \mu(s \in \mathbf{s}|s < e) \\ \Delta\sigma(\mathbf{s}) &= \sigma(\mathbf{s}) - \sigma(s \in \mathbf{s}|s < e) \\ \mathbf{s}_a &= \{s \in \mathbf{s}|s > e\} \\ \mathbf{E}_{seq} &= continuous\ anomalous\ sequence. \end{aligned} \tag{17}$$

In brief, NDT wants to find a threshold that causes the greatest decrease in the mean and standard deviation of $\mathbf{s}$ when all values greater than the threshold are removed. Moreover, because anomalies are sparse, NDT also considers preventing generating a large number of anomalous values ($|\mathbf{s}_a|$) and anomalous sequences ($|\mathbf{E}_{seq}^2|$).

6. Evaluation

Our evaluation seeks to answer the following research questions.

- **RQ1:** Does the data collected from the proposed network monitoring system work well on the detection of hidden network faults?
- **RQ2:** Does STCell-VAE outperform the approach that simply combines detection outcomes from a temporal-based method and a spatial-based method?
- **RQ3:** Can STCell-VAE outperform other anomaly detection methods?
- **RQ4:** Can STCell-VAE outperform other methods when the topology of the system dynamically changes?
- **RQ5:** How does each technique within STCell-VAE impact its performance?
- **RQ6:** How effective and efficient STCell-VAE is in a larger system?

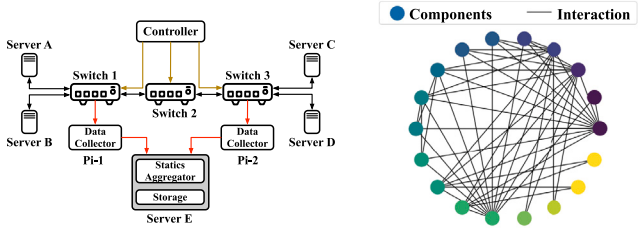
6.1. Experiments settings

To assess the effectiveness of our approach, this study conducts two types of experiments: simulation-based and real-world experiments.

6.1.1. Real-world experiments

Experimental Platform. In the real-world experiments, we have built a physical testbed based on SDN whose topology is shown in Fig. 10. This physical testbed comprises four physical servers connected through three H3C S5560X-EI SDN switches, and it utilizes an OpenFlow controller named Ryu [75] to distribute flow table entries. For the network monitoring system, For the network monitoring system, we installed a set of *Data Collectors* on multiple Raspberry Pis (i.e., embedded device based on ARM), each directly connected to the switches. These collectors are implemented in C++. Additionally, the *Statistics Aggregator* is implemented as a Python HTTP server and both the *Statistics Aggregator* and *Storage* are all running on the same server.

Then we deploy a virtualized distributed application named HipsterShop [76] on the testbed. HipsterShop is a web-based application widely used as a benchmark for evaluating different algorithms



(a) **The topology of the testbed.** Brown lines represent Openflow rules sent from controller to switches. Red lines represent matched packets which are analysed.

(b) **The topology of the deployed application.** Components belong to the same service are plotted in the same color.

Fig. 10. The topology of the physical testbed and the deployed application.

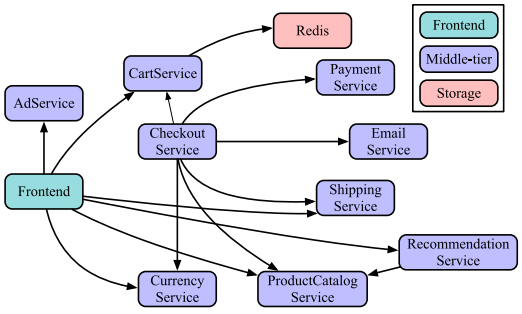
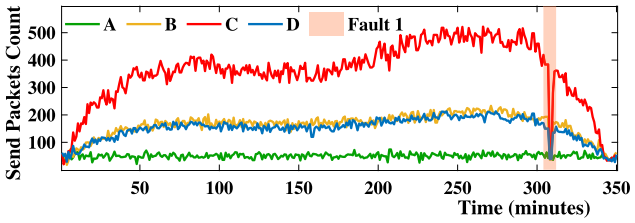
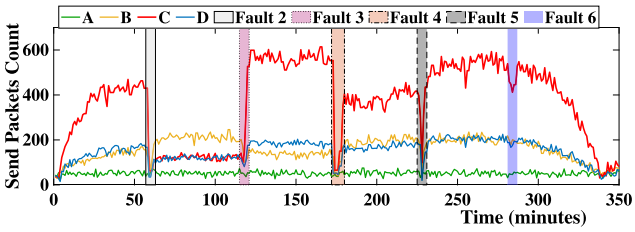


Fig. 11. Hipster Shop microservice architecture. The requests generated by clients are issued to Frontend services. Then, the requests are delivered to different back-end services according to the types of requests .



(a) **Time series with single point anomaly which is highlighted in pink shadow.**



(b) **Time series with trend anomalies and a point anomaly, which are highlighted in different colors.**

Fig. 12. An example of two types of anomalies. *Fault 1* in (a) and *fault 6* in (b) are point anomalies. Other faults in (b) are trend anomalies.

Table 2
Descriptions of real-world experimental datasets.

	sFlow-metrics	Component-metrics	Scaling-metrics
Total observations (T)	5291	5291	12 960
Total periods	15	15	29
Total components (N)	4	18	40 (19 original)
Metrics per component (F)	2	4	4
Total scaling events	0	0	24
Size	0.27 MB	2.8 MB	5.82 MB
Anomaly ratio	4.45%	4.45%	2.31%

and designs in microservice systems. As Fig. 11 shows, it consists of 11 different microservices interconnected with gRPC protocol. The component-level topology of the application in this experiment is depicted in Fig. 10(b). To emulate real-world scenarios, we generate a workload for the deployed application based on open-source production data from Taobao [77], which exhibits a daily periodic pattern. To expedite data collection efficiency, we condense it into a half-day periodic pattern.

Fault Injection. In these experiments, we use a fault injection tool named Chaosblade [78] to inject CPU exhaustion fault, memory exhaustion fault and network fault to the selected components or the selected servers in the testbed. Specifically, the injected network faults include network packet drop and packet delay. The first is to inject network loss fault to mimic connectivity failures and the second is to randomly delay the packet to mimic network jams. Each injection lasts for 3 min, and the interval between each fault injection is greater than 45 min.

As shown in Fig. 12, these faults have different effects, classified as **point anomaly** and **trend anomaly**. A point anomaly causes time series to deviate from normal in a few time points. A trend anomaly results in the trend change of time series. The trend change always lasts for a long period ($\geq$ 45 time points) until another network fault occurs.

Dynamic Changes of Application. To evaluate the effectiveness of our method in detecting anomalies when the network topology changes, we intermittently scale different services of the running application in or out. In particular, the number of service instances will decrease or increase during the experiments.

Datasets. We have generated three datasets in real-world experiments, namely, *sFlow-metrics*, *Components-metrics*, and *Scaling-metrics*. The details of these datasets are shown in Table 2.

sFlow-metrics dataset is generated using sFlow [61], which monitors network from a perspective of the physical network. This dataset comprises only four components because sFlow lacks the capability to analyze virtual network flows in detail, limiting its capability to extract network metrics for the application components. sFlow aggregates the number of sent and received packets for each server per minute. Consequently, the *sFlow-metrics* dataset includes two metrics (*sent packet counts per minute* and *received packet counts per minute*) for four servers. In contrast, both the *Component-metrics* and *Scaling-metrics* datasets are generated using our network monitoring system, which provides a finer-grained view of the network. As a result, these datasets encompass metrics from all application components and contain four metrics (*sent packet counts per minute*, *received packet counts per minute*, *sent packet byte counts per minute*, and *received packet byte counts per minute*). Specifically, the *Scaling-metrics* dataset includes 24 scaling events, where the application, consisting of 19 components, scales in and out to an application with 40 components. During the experiment to generate *Component-metrics*, only the network faults are injected to the selected application components. In contrast, in the experiment to generate *Scaling-metrics*, various types of faults are injected into both application components and physical nodes.

6.1.2. Simulation experiments

Due to constraints imposed by our limited hardware resources, it is not feasible to deploy a network system with sufficient size to fully validate the scalability of our approach. Consequently, we employ simulations to evaluate the scalability of the proposed detection method across various large-scale simulated networks. In simulation experiments, we utilize a popular network emulator, Mininet [41], which is capable to quickly create a virtual network, running actual kernel, switch and software code on a single machine. We simulate four of the most popular networks selected from [79], which are widely used in academic [80–83]. Table 3 presents the number of nodes and edges of the selected networks. We choose multiple pairs of source and

Table 3
Description of simulation experimental datasets.

	Bell Canada (BC)	US signal (US)	VTLWavenet (VW)	TATA (TT)
Total components (N)	48	61	92	145
Total Edges	130	158	192	388
Total observations (T)	7200	7200	7200	7200
Total periods	16	16	16	16
Metrics per component (F)	4	4	4	4
Size	8.1 MB	11.6 MB	16.8 MB	23.2 MB
Anomaly ratio	1.25%	1.25%	1.25%	1.25%

destination nodes, each generating a consecutive background flow. The packet size and inter-packet gap of these background flow are consistent with the distribution of the generated workload in the real-world experiments.

Datasets. For this evaluation, we monitor the total packet count and the total byte count sent and received by all the nodes. Large network simulations are all running for 2 days. During simulation, a Linux traffic control tool TC [84] is used to randomly inject two types of network faults (i.e. packet delay and drop) to the simulated switch's interface(s).

6.2. Performance metrics

Recent proposed anomaly detection methods always use classical Precision and Recall to evaluate the performance. However, classical metrics are defined for set of independent time points and they cannot capture time series specific characteristics. Therefore, they mislead the effectiveness of anomaly detection algorithms. As stated in [85], many real-world anomalies are range-based, indicating that they last for a period of time. Therefore, range-based Precision and Recall in [85] is used as the performance metrics.

Range-based metrics are the extension of classical metrics, while satisfying demands of range-based detection evaluation. The first demand is whether the algorithm catches the *existence* of an anomaly. The second demand (*cardinality*) cares about whether each independent anomaly range is detected as a single unit. Other demands consider the overlap between the predicted anomaly range and the real one. The third demand quantifies how large the *size* of the overlap is. The fourth demand considers *position* of the overlap in different scenarios.

Given a set of real anomaly ranges $R = \{R_1, \ldots, R_{N_r}\}$ and a set of predicted anomaly ranges $P = \{P_1, \ldots, P_{N_p}\}$. The calculation of $Recall(R, P)$ is shown as Eq. (18). The computation of $Recall_T(R_i, P)$ measures *existence*, *cardinality*, *size*, *position* stated as above. Specifically, *Position* measurement methods includes Flat, Front-end, Back-end and Middle. If all index positions of an anomaly range are equally important, the Flat method should be chosen. If earlier (later) index positions of an anomaly range are more important, the Front-end (Back-end) method should be used. The Middle method is used to prioritize the middle part of anomaly ranges. The choice for them depends on the scenario in anomaly detection.

$$Recall(R, P) = \frac{\sum_{i=1}^{N_r} Recall_T(R_i, P)}{N_r} \tag{18}$$

The calculation of $Precision(R, P)$ is shown as Eq. (19), which is similar to the calculation of $Recall(R, P)$. After calculating $Precision(R, P)$ and $Recall(R, P)$, the range based $F_1 - score$ is the same as the classical metrics.

$$Precision(R, P) = \frac{\sum_{i=1}^{N_p} Precision_T(R, P_i)}{N_p} \tag{19}$$

6.3. Experiment setup

Baseline models used in this study include univariate detectors and multivariate detectors. For univariate detectors, a traditional method **ARIMA** [86] and a state-of-the-art method **Donut** [19] are used. The detection is performed on each time series metrics individually and the best result is chosen as the final evaluation. Multivariate detectors include previous methods (**PCA** [87], **LOF** [88]) and deep learning (DL) models (**LSTM-AD** [23], **LSTM-ED** [24], **OmniAnomaly** [25], **RANSynCoders** [27]). For previous methods, we concatenate metrics from different components into a matrix and feed the matrix into detectors for evaluation. For DL models, the detection is performed on each component one by one, and the average anomaly score of all components is used as the score for evaluation. Since not all the baseline models provide a mechanism to select thresholds, we use the same dynamic threshold selection strategy, NDT, to evaluate the actual performance of all models in practice.

We set sliding window size W as 10, which spans 10 min in all datasets. We set the hidden state dimension S_h per component to 2, the learning rate to 10^{-3}, the batch size to 256, the sampling number L to 20, the size of **s** (S) to 45. All the parameters can be tuned for better performance.

In experiments, we use metrics from the first four periods as training data and metrics from other periods as testing data. Most existing anomaly detectors assume that the training data is not contaminated, but this is hard to guarantee in reality. Therefore, we do not have such an assumption in our approach. We train the model and other detectors on a server with one NVIDIA Tesla P100-SXM2 GPU.

Table 4
Detection results on *sFlow-metrics* and *Component-metrics*.

Dataset	Precision	Recall	F1-Score
sFlow-metrics	0.092	0.2	0.126
Components-metrics	0.532	0.555	0.543

6.4. Comparisons in different granularities

Before evaluating the proposed model from different aspects, an experiment is presented to validate the effectiveness of the proposed network monitoring system in detecting hidden network faults. Since the topology of *sFlow-metrics* is trivial, we use a simple detector **LSTM-ED** which does not consider the topology to detect anomaly on *Components-metrics* and *sFlow-metrics*.

Table 4 shows the detection results in two datasets. The performance on *Components-metrics* significantly outperforms that on *sFlow-metrics*. This observation underscores the difficulty of detecting anomalies using metrics collected from traditional monitoring method. The challenge arises from the fact that a quintuple-defined flow typically comprises multiple application-specific flows, and faults within the application may not necessarily impact the metrics of quintuple-defined flows. This confirms the importance of monitoring the network at a finer granularity, as it enables network operators to comprehensively observe network.

6.5. Comparisons with simple combination

In this evaluation, our objective is to determine whether STCell-VAE demonstrates better performance compared to a fundamental amalgamation of detection outcomes derived from a temporal-based method and a spatial-based method. We implement a spatial-based detection method (S) and a temporal-based detection method (T), then we use them to detect anomalies on *Component-metrics*. These two methods use a network architecture (VAE) that is similar as STCell-VAE. S uses GCN model as the basic block to capture the spatial relationship while T uses LSTM model as the basic block to capture the temporal relationship. Then, both methods calculate anomaly scores for each individual observation following the same procedure described in Section 5.3.1. Subsequently, we combine results from two methods with four operators, namely *Union*, *Intersection*, *Mean* and *Max*. For *Union* operator, we use the anomaly score and the NDT method to derive

Table 5
Comparison of different anomaly detectors.

Detectors	Flat			Front-end			Middle			Back-end		
	Precision	Recall	F1	Precision	Recall	F1	Precision	Recall	F1	Precision	Recall	F1
ARIMA [86]	0.414	0.465	0.438	0.423	0.480	0.450	0.414	0.473	0.442	0.414	0.451	0.432
Donut [19]	0.682	0.561	0.616	0.682	0.551	0.610	0.682	0.572	0.622	0.682	0.572	0.622
PCA [87]	0.292	0.345	0.316	0.292	0.321	0.306	0.292	0.356	0.321	0.292	0.369	0.326
LOF [88]	0.08	0.608	0.132	0.07	0.617	0.132	0.07	0.608	0.132	0.07	0.598	0.132
LSTM-ED [24]	0.553	0.561	0.557	0.553	0.561	0.557	0.553	0.576	0.564	0.553	0.561	0.557
LSTM-AD [23]	0.240	0.381	0.294	0.240	0.391	0.300	0.240	0.379	0.293	0.240	0.370	0.291
OmniAnomaly [25]	0.367	0.322	0.343	0.367	0.305	0.333	0.367	0.329	0.347	0.367	0.338	0.352
RANSynCoders [27]	0.194	0.457	0.272	0.194	0.468	0.274	0.194	0.460	0.273	0.194	0.446	0.270
STCell-VAE	0.785	0.667	0.721	0.785	0.659	0.716	0.785	0.685	0.731	0.785	0.674	0.726

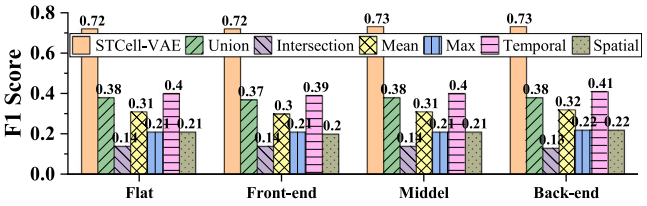


Fig. 13. F1 scores obtained by STCell-VAE, the temporal-based method, the spatial-based method and other methods based on their simple combination.

labels of S and T individually. These labels are then merged using a union operation. For *Intersection* operator, the derived labels are merged using an intersection operation. For *Mean* operator, we calculate a final score for each observation by averaging scores from both methods. Then the final scores are used to derive the labels. *Max* means that the final scores are combined by selecting the maximum scores for each observation.

Fig. 13 illustrates the F1 scores derived from STCell-VAE, T, S and four alternative methods relying on the basic fusion of T and S. As it shows, S underperforms T, which in turn exhibits inferior performance to STCell-VAE. This observation confirms the significance of temporal relationships in anomaly detection. While certain faults might manifest as disruptions in the spatial relationships within metrics, they typically necessitate time to progressively manifest in the network. Therefore, it is imperative that training involving spatial relationships not disregard the consideration of temporal relationships. This underlying principle explains the comparatively weaker performance of S. Attributed to the limitation of S and the requirement of combination methods for a certain level of comparability between combined detectors, four basic combination methods significantly underperforms STCell-VAE and even underperforms T. On the contrary, STCell-VAE integrates both spatial and temporal relationships during the training process, which contributes to its superior performance.

6.6. Overall performance

Table 5 shows the detection results of different detectors in different scenarios on *Components-metrics* dataset. As we can see, all traditional multivariate anomaly detectors (e.g. **LOF**, **PCA**) do not perform well. This is primarily because the performance of these methods relies on the satisfaction of their assumptions regarding data distribution. When these assumptions are violated, their performance degrades. In our experiments, a real-world workload is generated for the application, leading to dynamic changes in data distribution and thus, violation of the assumptions. Therefore, traditional multivariate anomaly detectors exhibit poor performance in our experiments.

Univariate anomaly detectors such as **ARIMA** and **Donut** perform better than some multivariate anomaly detectors in our experiment. This is attributed to the fact that certain individual time series metrics in our dataset provide critical indications of faults. Univariate detectors works well when trained on such insightful time series metrics. However, it remains challenging to manually select an appropriate time series metric in advance without a priori knowledge of the specific application and its behavior.

LSTM-AD, **LSTM-ED** and **OmniAnomaly** seem not perform well in our experiment because the models are trained without combining the spatial dependence. Their performance suffers due to the absence of spatial information. **RANSynCoders** also shows poor performance in this study. This is because **RANSynCoders** utilizes a network to capture trends in time series and generate a representation for multiple time series. However, these trends in time series may be similar due to comparable business behaviors. With the consideration of spatial information and temporal information, STCell-VAE outperforms other baseline models.

6.7. Performance in dynamic network

In this evaluation, our objective is to determine whether STCell-VAE surpasses other detection methods when the topology of the application dynamically changes. We choose four better detection models (i.e., **Donut**, **OmniAnomaly**, **ARIMA**, **LSTM-ED**) as the baselines. Notably, these models do not inherently support detection in such systems where the topology dynamically changes, unless the models are retrained when the system changes. To address this limitation, we implemented a strategy to render these baseline models to detect anomalies in the dynamic network. In details, when a new component is added, we measure the similarity in metrics between the new component and the original components. Subsequently, we utilize the trained model of the most similar original component as the model for the new component.

Table 6 shows the evaluation results. As we can see, the performance of all these models declines when the number of application components and the application's topology change. This observation proves that the dynamic changes in the application can indeed impact anomaly detection performance. The addition or removal of components will disrupt the network behavior patterns of the original components, thereby hindering these models from accurate detection. However, STCell-VAE continues to outperform other models, affirming that integrating spatial and temporal relationships is more beneficial than relying solely on temporal information when the topology changes.

6.8. Design choices validation

6.8.1. Effect of alarm policy tuning

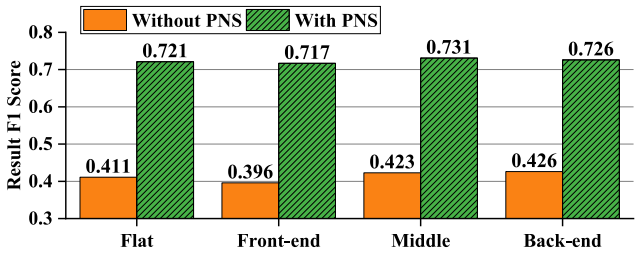
In practice, we have observed that most anomalies last for a period of time. Therefore, we tune the alarm policy for the proposed model and other baseline models. Only the predicted N consecutive anomalous points will raise an alarm and the filter anomalous points are considered as normal (N is set as 3 in this study).

Table 6
Comparison of different anomaly detectors in the system whose topology dynamically changes.

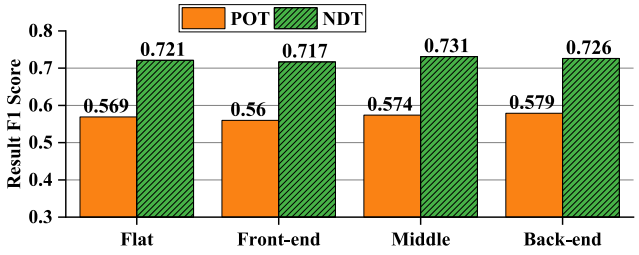
Detectors	Flat			Front-end			Middle			Back-end		
	Precision	Recall	F1	Precision	Recall	F1	Precision	Recall	F1	Precision	Recall	F1
ARIMA [86]	0.123	0.431	0.191	0.123	0.439	0.193	0.123	0.441	0.193	0.123	0.424	0.191
Donut [19]	0.790	0.439	0.565	0.790	0.425	0.553	0.790	0.457	0.579	0.790	0.453	0.576
LSTM-ED [24]	0.337	0.315	0.326	0.337	0.323	0.330	0.337	0.325	0.331	0.337	0.307	0.321
OmniAnomaly [25]	0.349	0.431	0.192	0.349	0.267	0.303	0.349	0.292	0.318	0.349	0.302	0.324
STCell-VAE	0.575	0.676	0.621	0.575	0.648	0.609	0.575	0.694	0.629	0.575	0.704	0.632

Table 7
F1-Score obtained with and without alarm policy tuning.

Method	Flat	Front-end	Middle	Back-end
without Filter	0.609	0.608	0.617	0.611
With Filter	0.721	0.716	0.731	0.756



(a) **F1-score obtained with and without PNS**



(b) **F1-score obtain with POT and NDT**

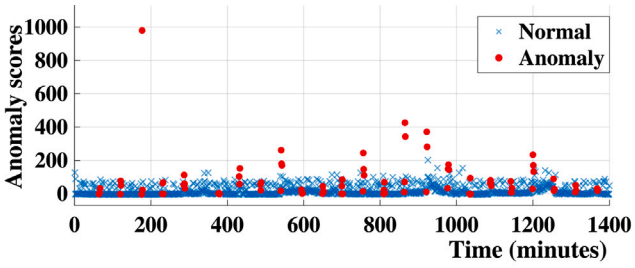
Fig. 14. F1-score in different scenarios.

Table 7 shows the F1-score obtained by the model with and without alarm policy tuning. The model with filtering can help to achieve higher F1-score, which means that network operators' work can be significantly reduced through eliminating false positives.

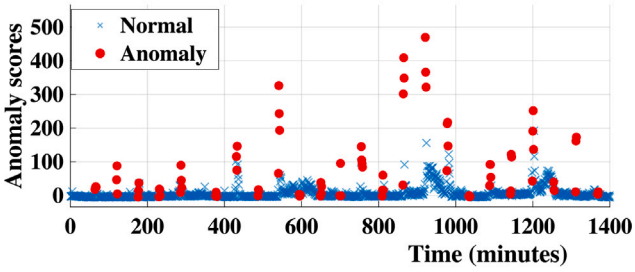
6.8.2. *Effect of periodic noises smoothing*

In this evaluation, we prove the necessity of PNS by comparing our model with and without PNS.

The F1-scores obtained by STCell-VAE with and without PNS are shown in Fig. 14(a). STCell-VAE with PNS outperforms STCell-VAE without PNS. The reason behind can be presented with Fig. 15, which shows the distribution of anomaly scores with and without PNS. As Fig. 15(a) shows, some anomalous observations are considered normal since anomaly scores of all observations vary a lot. This is because that the existence of the periodic noises hinders the modeling of the normal pattern. In contrast, these periodic noises are removed, rendering normal observations to be reconstructed better. Therefore, anomalous observations are distinguished from normal observations and more anomalous observations are detected, which can be seen from Fig. 15(b). This demonstrates the capability of STCell-VAE with PNS to be resilient to some unexpected noises.

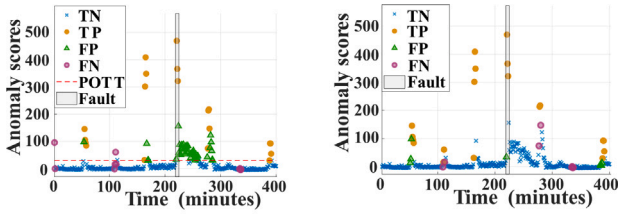


(a) **Without PNS**



(b) **With PNS**

Fig. 15. The distribution of anomaly scores with and without Periodic Noises Smoothing (PNS).



(a) **POT** (b) **NDT**

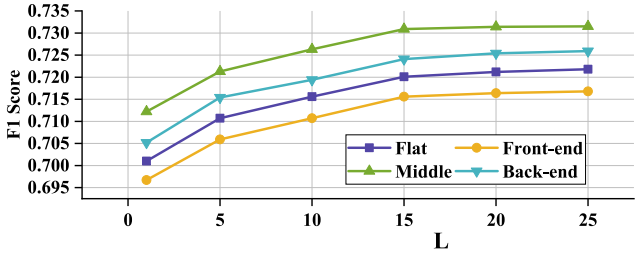
Fig. 16. The distribution of anomaly scores using different threshold selection methods. *POT T* represents the threshold selected by POT. *Fault* represents an anomaly example.

6.8.3. *Effect of dynamic threshold selection*

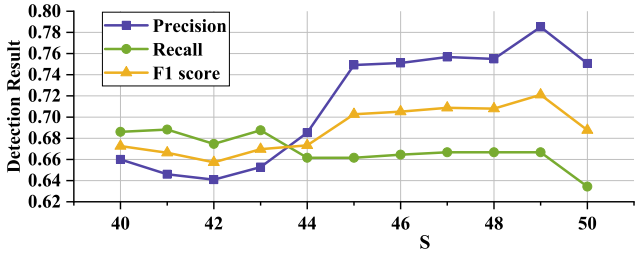
In this section, we will prove the necessity of a dynamic threshold selection method in anomaly detection. We compare NDT used in our model with a popular static threshold selection method (POT [25]). We tune S of NDT and choose the best threshold of POT to achieve their best performance.

F1-scores obtained by these two methods are presented in Fig. 14(b), showing that NDT outperforms POT in all scenarios.

According to Section 6.3, if a trend anomaly occurs, the new trend of time series metrics would cause higher anomaly scores even after the anomaly is over. For example, in Fig. 16, a trend anomaly *Fault* raises the shift of trend in all metrics. After *Fault*, we can observe that anomaly scores of the following normal time points are still higher. If a static threshold selection method is used, these time points will be considered as anomalies and raise false positives. However, if NDT is used, the

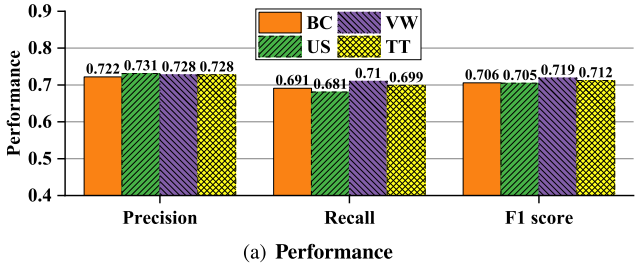


(a) **Impact of L on F1-score in different scenarios**

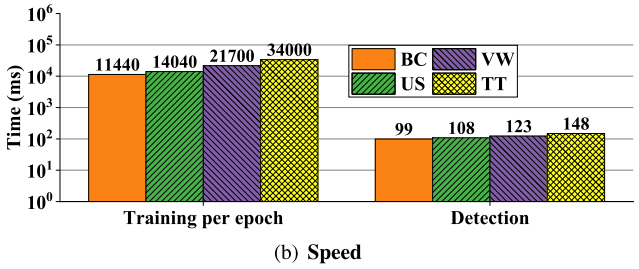


(b) **Impact of S on detection result in Flat scenario**

Fig. 17. Impact of different parameters.



(a) **Performance**



(b) **Speed**

Fig. 18. The model's performance and speed in simulated systems.

threshold can be dynamically selected according to the distribution of anomaly scores. Thus, NDT avoids considering these time points as anomalies and reduces false positives. It can be illustrated clearly in Fig. 16. The number of TP (True Positive) and FN (False Negative) are the same in POT and NDT (19 and 11). However, the number of FP (False Positive) in POT is larger than in NDT (44 and 8, respectively) while the number of TN (True Negative) in POT is smaller than in NDT (327 and 363, respectively). This observation supports the necessity of NDT in our model.

6.9. Parameter comparison

6.9.1. The impact of L

Fig. 17(a) shows how sampling number L in reconstruction probability calculation impacts the effectiveness of STCell-VAE. From Fig. 17(a), we can see that F1-score gradually stabilizes as L increases from 0 to 25. Before L reaches 15, the F1-score increases and after that it becomes stable. The result indicates that the model performs better by calculating reconstruction probability with a larger sampling number. However, over-sampling introduces extra overhead and is not helpful in improving performance. In this study, we suggest L is set as 20.

6.9.2. The impact of S

Fig. 17(b) shows how the size of s, S, in selecting threshold impacts effectiveness of STCell-VAE. Because the interval between each injection is greater than 45 min, we set S to values range from 40 to 50 and compare the performance metrics of them in Flat scenario. As we can see, the performance of the model remains unchanged when S is from 40 to 43, but then precision and F1 dramatically increase when S is from 43 to 45. After S is larger than 45, the performance remains the same. STCell-VAE gets better performance if S is larger than the fault interval. The observation says, setting S to a value slightly larger than the fault interval helps to increase precision by reducing false positives. In this study, S is preset as 45.

6.10. Model scalability

Finally, we evaluate STCell-VAE's scalability by simulating four different large-scale network, whose details are described in Table 3. We first validate the detection performance of the proposed algorithm in different scale networks. Fig. 18(a) shows the performance of the proposed algorithm in different scale networks. As the scale of the network topology increases from 48 to 145, the performance of the algorithm keeps stable and consistent. Therefore, we conclude that the network scale has little influence on the performance of the algorithm. In the meantime, we also note that the training time and the detection time increases slightly as the scale of the network increases from Fig. 18(b). As the scale of TT is 1.64× bigger than BC, the training time increases by 2.97× while the detection time only increases by 1.49×. This is because the number of edges between nodes becomes larger as the scale of topology grows, which needs more time to learn the relationship between nodes in the training phase. The result shows that the trained model still detects timely in bigger network.

7. Conclusion

We have presented a network monitoring system that provides extra visibility to the network status of application-specific flows, which helps network operators obtain deep insights about the network status. With the monitoring data, we propose a robust anomaly detection method to detect the hidden faults exist in the application. The algorithm goes beyond detecting failures by using STCell-VAE to model the system in a more comprehensive view through combining the temporal and spatial information. Moreover, the proposed method is resilient to periodic noises and can still be updated and incorporate new network information in a system whose topology dynamically changes. With the experiments on four simulated large-scale networks and on a real-world testbed, we have proven the effectiveness of our monitoring system and the proposed anomaly detection algorithm.

Acknowledgement

We greatly appreciate the insightful feedback from the anonymous reviewers. The research is supported by the National Key Research and Development Program of China (2019YFB1804002).

CRediT authorship contribution statement

Hongyang Chen: Conceptualization, Methodology, Software, Data curation, Investigation, Formal analysis, Writing – original draft. **Pengfei Chen:** Conceptualization, Funding acquisition, Resources, Supervision,

Writing – review & editing. **Benran Wang:** Investigation, Data curation, Visualization. **Xian Yu:** Writing – review & editing, Validation. **Xiaofan Chen:** Writing – review & editing. **Dandan Ma:** Writing – review & editing. **Zibin Zheng:** Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data available at:

Application Metrics monitoring by SDN based monitor system (Original data) (github)

.

References

[1] C. Zhang, X. Peng, C. Sha, K. Zhang, Z. Fu, X. Wu, Q. Lin, D. Zhang, DeepTraLog: Trace-log combined microservice anomaly detection through graph-based deep learning, in: Proceedings of the 44th International Conference on Software Engineering, ICSE '22, Association for Computing Machinery, New York, NY, USA, 2022, pp. 623–634, http://dx.doi.org/10.1145/3510003.3510180.

[2] Docker, https://www.docker.com/.

[3] Kubernetes, https://kubernetes.io/.

[4] Docker Swarm, https://docs.docker.com/engine/swarm/.

[5] H. Zhou, M. Chen, Q. Lin, Y. Wang, X. She, S. Liu, R. Gu, B.C. Ooi, J. Yang, Overload control for scaling WeChat microservices, in: Proceedings of the ACM Symposium on Cloud Computing, SoCC '18, Association for Computing Machinery, New York, NY, USA, 2018, pp. 149–161, http://dx.doi.org/10.1145/3267809.3267823.

[6] Flannel, https://github.com/flannel-io/flannel.

[7] Calico, https://github.com/projectcalico/calico.

[8] N. Handigol, B. Heller, V. Jeyakumar, D. Mazières, N. McKeown, I know what your packet did last hop: using packet histories to troubleshoot networks, in: Proceedings of the 11th USENIX Conference on Networked Systems Design and Implementation, NSDI'14, USENIX Association, USA, 2014, pp. 71–85.

[9] J. Rasley, B. Stephens, C. Dixon, E. Rozner, W. Felter, K. Agarwal, J. Carter, R. Fonseca, Planck: Millisecond-scale monitoring and control for commodity networks, SIGCOMM Comput. Commun. Rev. 44 (4) (2014) 407–418, http://dx.doi.org/10.1145/2740070.2626310.

[10] Y. Zhu, N. Kang, J. Cao, A. Greenberg, G. Lu, R. Mahajan, D. Maltz, L. Yuan, M. Zhang, B.Y. Zhao, H. Zheng, Packet-level telemetry in large datacenter networks, SIGCOMM Comput. Commun. Rev. 45 (4) (2015) 479–491, http://dx.doi.org/10.1145/2829988.2787483.

[11] J. Suh, T.T. Kwon, C. Dixon, W. Felter, J. Carter, OpenSample: A low-latency, sampling-based measurement platform for commodity SDN, in: 2014 IEEE 34th International Conference on Distributed Computing Systems, 2014, pp. 228–237, http://dx.doi.org/10.1109/ICDCS.2014.31.

[12] A. Tootoonchian, M. Ghobadi, Y. Ganjali, OpenTM: Traffic matrix estimator for OpenFlow networks, in: Proceedings of the 11th International Conference on Passive and Active Measurement, PAM '10, Springer-Verlag, Berlin, Heidelberg, 2010, pp. 201–210.

[13] N.L.M. van Adrichem, C. Doerr, F.A. Kuipers, OpenNetMon: Network monitoring in OpenFlow software-defined networks, in: 2014 IEEE Network Operations and Management Symposium, NOMS, 2014, pp. 1–8, http://dx.doi.org/10.1109/NOMS.2014.6838228.

[14] G. Yu, P. Chen, H. Chen, Z. Guan, Z. Huang, L. Jing, T. Weng, X. Sun, X. Li, MicroRank: End-to-end latency issue localization with extended spectrum analysis in microservice environments, in: Proceedings of the Web Conference 2021, WWW '21, Association for Computing Machinery, New York, NY, USA, 2021, pp. 3087–3098, http://dx.doi.org/10.1145/3442381.3449905.

[15] J. Lin, P. Chen, Z. Zheng, Microscope: Pinpoint performance issues with causal graphs in micro-service environments, in: C. Pahl, M. Vukovic, J. Yin, Q. Yu (Eds.), Service-Oriented Computing - 16th International Conference, ICSOC 2018, Hangzhou, China, November 12–15, 2018, Proceedings, in: Lecture Notes in Computer Science, vol. 11236, Springer, 2018, pp. 3–20, http://dx.doi.org/10.1007/978-3-030-03596-9_1.

[16] G. Yu, P. Chen, Z. Zheng, Microscaler: Automatic scaling for microservices with an online learning approach, in: 2019 IEEE International Conference on Web Services, ICWS, 2019, pp. 68–75, http://dx.doi.org/10.1109/ICWS.2019.00023.

[17] B. Arzani, S. Ciraci, B.T. Loo, A. Schuster, G. Outhred, Taking the blame game out of data centers operations with NetPoirot, in: Proceedings of the 2016 ACM SIGCOMM Conference, SIGCOMM '16, Association for Computing Machinery, New York, NY, USA, 2016, pp. 440–453, http://dx.doi.org/10.1145/2934872.2934884.

[18] L. Castanheira, A. Schaeffer-Filho, T.A. Benson, Foxhound: Server-grade observability for network-augmented applications, in: Proceedings of the Eighteenth European Conference on Computer Systems, EuroSys '23, Association for Computing Machinery, New York, NY, USA, 2023, pp. 18–32, http://dx.doi.org/10.1145/3552326.3567502.

[19] H. Xu, W. Chen, N. Zhao, Z. Li, J. Bu, Z. Li, Y. Liu, Y. Zhao, D. Pei, Y. Feng, J. Chen, Z. Wang, H. Qiao, Unsupervised anomaly detection via variational auto-encoder for seasonal KPIs in web applications, in: Proceedings of the 2018 World Wide Web Conference, WWW '18, International World Wide Web Conferences Steering Committee, Republic and Canton of Geneva, CHE, 2018, pp. 187–196, http://dx.doi.org/10.1145/3178876.3185996.

[20] A. Siffer, P.-A. Fouque, A. Termier, C. Largouet, Anomaly detection in streams with extreme value theory, in: Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, KDD '17, Association for Computing Machinery, New York, NY, USA, 2017, pp. 1067–1075, http://dx.doi.org/10.1145/3097983.3098144.

[21] Z. Li, W. Chen, D. Pei, Robust and unsupervised KPI anomaly detection based on conditional variational autoencoder, in: 2018 IEEE 37th International Performance Computing and Communications Conference, IPCCC, 2018, pp. 1–9, http://dx.doi.org/10.1109/PCCC.2018.8710885.

[22] W. Chen, H. Xu, Z. Li, D. Pei, J. Chen, H. Qiao, Y. Feng, Z. Wang, Unsupervised anomaly detection for intricate KPIs via adversarial training of VAE, in: IEEE INFOCOM 2019 - IEEE Conference on Computer Communications, 2019, pp. 1891–1899, http://dx.doi.org/10.1109/INFOCOM.2019.8737430.

[23] P. Malhotra, L. Vig, G. Shroff, P. Agarwal, et al., Long short term memory networks for anomaly detection in time series, in: ESANN, Vol. 89, 2015, pp. 89–94.

[24] I.-F. Kao, Y. Zhou, L.-C. Chang, F.-J. Chang, Exploring a Long Short-Term Memory based Encoder-Decoder framework for multi-step-ahead flood forecasting, J. Hydrol. 583 (2020) 124631, http://dx.doi.org/10.1016/j.jhydrol.2020.124631.

[25] Y. Su, Y. Zhao, C. Niu, R. Liu, W. Sun, D. Pei, Robust anomaly detection for multivariate time series through stochastic recurrent neural network, in: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, KDD '19, Association for Computing Machinery, New York, NY, USA, 2019, pp. 2828–2837, http://dx.doi.org/10.1145/3292500.3330672.

[26] J. Audibert, P. Michiardi, F. Guyard, S. Marti, M.A. Zuluaga, USAD: UnSupervised anomaly detection on multivariate time series, in: Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, KDD '20, Association for Computing Machinery, New York, NY, USA, 2020, pp. 3395–3404, http://dx.doi.org/10.1145/3394486.3403392.

[27] A. Abdulaal, Z. Liu, T. Lancewicki, Practical approach to asynchronous multivariate time series anomaly detection and localization, in: Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, KDD '21, Association for Computing Machinery, New York, NY, USA, 2021, pp. 2485–2494, http://dx.doi.org/10.1145/3447548.3467174.

[28] A. Roy, D. Bansal, D. Brumley, H.K. Chandrappa, P. Sharma, R. Tewari, B. Arzani, A.C. Snoeren, Cloud datacenter SDN monitoring: Experiences and challenges, in: Proceedings of the Internet Measurement Conference 2018, IMC '18, Association for Computing Machinery, New York, NY, USA, 2018, pp. 464–470, http://dx.doi.org/10.1145/3278532.3278572.

[29] C. Fang, H. Liu, M. Miao, J. Ye, L. Wang, W. Zhang, D. Kang, B. Lyv, P. Cheng, J. Chen, Vtrace: Automatic diagnostic system for persistent packet loss in cloud-scale overlay network, in: Proceedings of the Annual Conference of the ACM Special Interest Group on Data Communication on the Applications, Technologies, Architectures, and Protocols for Computer Communication, SIGCOMM '20, Association for Computing Machinery, New York, NY, USA, 2020, pp. 31–43, http://dx.doi.org/10.1145/3387514.3405851.

[30] P. Chen, Y. Qi, P. Zheng, D. Hou, CauseInfer: Automatic and distributed performance diagnosis with hierarchical causality graph in large distributed systems, in: IEEE INFOCOM 2014 - IEEE Conference on Computer Communications, 2014, pp. 1887–1895, http://dx.doi.org/10.1109/INFOCOM.2014.6848128.

[31] Z. Zheng, T.C. Zhou, M.R. Lyu, I. King, Component ranking for fault-tolerant cloud applications, IEEE Trans. Serv. Comput. 5 (4) (2012) 540–550, http://dx.doi.org/10.1109/TSC.2011.42.

[32] M. Zasadziński, M. Solé, A. Brandon, V. Muntés-Mulero, D. Carrera, Next stop "NoOps": Enabling cross-system diagnostics through graph-based composition of logs and metrics, in: 2018 IEEE International Conference on Cluster Computing, CLUSTER, 2018, pp. 212–222, http://dx.doi.org/10.1109/CLUSTER.2018.00039.

[33] C. Guo, Pingmesh: A large-scale system for data center network latency measurement and analysis, in: SIGCOMM, ACM, 2015.

[34] C. Tan, Z. Jin, C. Guo, T. Zhang, H. Wu, K. Deng, D. Bi, D. Xiang, NetBouncer: Active device and link failure localization in data center networks, in: 16th USENIX Symposium on Networked Systems Design and Implementation, NSDI 19, USENIX Association, Boston, MA, 2019, pp. 599–614.

[35] T. Ueda, T. Nakaike, M. Ohara, Workload characterization for microservices, in: 2016 IEEE International Symposium on Workload Characterization, IISWC, IEEE, 2016, pp. 1–10.

[36] S. Luo, H. Xu, K. Ye, G. Xu, L. Zhang, G. Yang, C. Xu, The power of prediction: Microservice auto scaling via workload learning, in: Proceedings of the 13th Symposium on Cloud Computing, SoCC '22, Association for Computing Machinery, New York, NY, USA, 2022, pp. 355–369, http://dx.doi.org/10.1145/3542929.3563477.

[37] A. Patcha, J.-M. Park, An overview of anomaly detection techniques: Existing solutions and latest technological trends, Comput. Netw. 51 (12) (2007) 3448–3470, http://dx.doi.org/10.1016/j.comnet.2007.02.001.

[38] V. Chandola, A. Banerjee, V. Kumar, Anomaly detection: A survey, ACM Comput. Surv. 41 (3) (2009) http://dx.doi.org/10.1145/1541880.1541882.

[39] M. Ahmed, A. Naser Mahmood, J. Hu, A survey of network anomaly detection techniques, J. Netw. Comput. Appl. 60 (2016) 19–31, http://dx.doi.org/10.1016/j.jnca.2015.11.016.

[40] M.H. Bhuyan, D.K. Bhattacharyya, J.K. Kalita, Network anomaly detection: Methods, systems and tools, IEEE Commun. Surv. Tutor. 16 (1) (2014) 303–336, http://dx.doi.org/10.1109/SURV.2013.052213.00046.

[41] B. Lantz, B. Heller, N. McKeown, A network in a laptop: Rapid prototyping for software-defined networks, in: Proceedings of the 9th ACM SIGCOMM Workshop on Hot Topics in Networks, in: Hotnets-IX, Association for Computing Machinery, New York, NY, USA, 2010, http://dx.doi.org/10.1145/1868447.1868466.

[42] S. Sadrhaghighi, M. Dolati, M. Ghaderi, A. Khonsari, FlowShark: Sampling for high flow visibility in SDNs, in: IEEE INFOCOM 2022 - IEEE Conference on Computer Communications, 2022, pp. 160–169, http://dx.doi.org/10.1109/INFOCOM48880.2022.9796658.

[43] V. Sekar, M.K. Reiter, H. Zhang, Revisiting the case for a minimalist approach for network flow monitoring, in: Proceedings of the 10th ACM SIGCOMM Conference on Internet Measurement, IMC '10, Association for Computing Machinery, New York, NY, USA, 2010, pp. 328–341, http://dx.doi.org/10.1145/1879141.1879186.

[44] W. Wu, K. He, A. Akella, PerfSight: Performance diagnosis for software dataplanes, in: Proceedings of the 2015 Internet Measurement Conference, IMC '15, Association for Computing Machinery, New York, NY, USA, 2015, pp. 409–421, http://dx.doi.org/10.1145/2815675.2815698.

[45] J.R. Ballard, I. Rae, Extensible and scalable network monitoring using OpenSAFE, in: 2010 Internet Network Management Workshop/Workshop on Research on Enterprise Networking, INM/WREN 10, USENIX Association, San Jose, CA, 2010.

[46] A. Dhamdhere, D.D. Clark, A. Gamero-Garrido, M. Luckie, R.K.P. Mok, G. Akiwate, K. Gogia, V. Bajpai, A.C. Snoeren, K. Claffy, Inferring persistent interdomain congestion, in: Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication, SIGCOMM '18, Association for Computing Machinery, New York, NY, USA, 2018, pp. 1–15, http://dx.doi.org/10.1145/3230543.3230549.

[47] H. Zeng, P. Kazemian, G. Varghese, N. McKeown, Automatic test packet generation, in: Proceedings of the 8th International Conference on Emerging Networking Experiments and Technologies, CoNEXT '12, Association for Computing Machinery, New York, NY, USA, 2012, pp. 241–252, http://dx.doi.org/10.1145/2413176.2413205.

[48] K. Agarwal, E. Rozner, C. Dixon, J. Carter, SDN traceroute: Tracing SDN forwarding without changing network behavior, in: Proceedings of the Third Workshop on Hot Topics in Software Defined Networking, HotSDN '14, Association for Computing Machinery, New York, NY, USA, 2014, pp. 145–150, http://dx.doi.org/10.1145/2620728.2620756.

[49] O. Tilmans, T. Bühler, I. Poese, S. Vissicchio, L. Vanbever, Stroboscope: Declarative network monitoring on a budget, in: 15th USENIX Symposium on Networked Systems Design and Implementation, NSDI 18, USENIX Association, Renton, WA, 2018, pp. 467–482.

[50] G. Yang, H. Jin, M. Kang, G. Jun Moon, C. Yoo, Network monitoring for SDN virtual networks, in: IEEE INFOCOM 2020 - IEEE Conference on Computer Communications, 2020, pp. 1261–1270, http://dx.doi.org/10.1109/INFOCOM41043.2020.9155260.

[51] G. Liu, M. Trotter, Y. Ren, T. Wood, NetAlytics: Cloud-scale application performance monitoring with SDN and NFV, in: Proceedings of the 17th International Middleware Conference, Middleware '16, Association for Computing Machinery, New York, NY, USA, 2016, http://dx.doi.org/10.1145/2988336.2988344.

[52] A. Khurshid, W. Zhou, M. Caesar, P.B. Godfrey, Veriflow: Verifying network-wide invariants in real time, SIGCOMM Comput. Commun. Rev. 42 (4) (2012) 467–472, http://dx.doi.org/10.1145/2377677.2377766.

[53] N. Handigol, B. Heller, V. Jeyakumar, D. Maziéres, N. McKeown, Where is the debugger for my software-defined network? in: Proceedings of the First Workshop on Hot Topics in Software Defined Networks, HotSDN '12, Association for Computing Machinery, New York, NY, USA, 2012, pp. 55–60, http://dx.doi.org/10.1145/2342441.2342453.

[54] A. Wundsam, D. Levin, S. Seetharaman, A. Feldmann, Ofrewind: Enabling record and replay troubleshooting for networks, in: Proceedings of the 2011 USENIX Conference on USENIX Annual Technical Conference, USENIXATC '11, USENIX Association, USA, 2011, p. 29.

[55] Q. Huang, X. Jin, P.P.C. Lee, R. Li, L. Tang, Y.-C. Chen, G. Zhang, SketchVisor: Robust network measurement for software packet processing, in: Proceedings of the Conference of the ACM Special Interest Group on Data Communication, SIGCOMM '17, Association for Computing Machinery, New York, NY, USA, 2017, pp. 113–126, http://dx.doi.org/10.1145/3098822.3098831.

[56] Y. Zhao, K. Yang, Z. Liu, T. Yang, L. Chen, S. Liu, N. Zheng, R. Wang, H. Wu, Y. Wang, N. Zhang, LightGuardian: A Full-Visibility, lightweight, in-band telemetry system using sketchlets, in: 18th USENIX Symposium on Networked Systems Design and Implementation, NSDI 21, USENIX Association, 2021, pp. 991–1010.

[57] W. Wu, G. Wang, A. Akella, A. Shaikh, Virtual network diagnosis as a service, in: Proceedings of the 4th Annual Symposium on Cloud Computing, SOCC '13, Association for Computing Machinery, New York, NY, USA, 2013, http://dx.doi.org/10.1145/2523616.2523621.

[58] B. Schölkopf, R. Williamson, A. Smola, J. Shawe-Taylor, J. Platt, Support vector method for novelty detection, in: Proceedings of the 12th International Conference on Neural Information Processing Systems, NIPS '99, MIT Press, Cambridge, MA, USA, 1999, pp. 582–588.

[59] Y. Li, B. Fang, L. Guo, Y. Chen, Network anomaly detection based on TCM-KNN algorithm, in: Proceedings of the 2nd ACM Symposium on Information, Computer and Communications Security, ASIACCS '07, Association for Computing Machinery, New York, NY, USA, 2007, pp. 13–19, http://dx.doi.org/10.1145/1229285.1229292.

[60] D.P. Kingma, M. Welling, Auto-encoding variational bayes, 2013, arXiv preprint arXiv:1312.6114.

[61] P. Phaal, S. Panchen, N. McKee, Inmon corporation's sflow: A method for monitoring traffic in switched and routed networks, 2001, RFC Editor, Internet Eng. Task Force, RFC 3176, USA.

[62] M. Ma, S. Zhang, J. Chen, J. Xu, H. Li, Y. Lin, X. Nie, B. Zhou, Y. Wang, D. Pei, Jump-Starting multivariate time series anomaly detection for online service systems, in: 2021 USENIX Annual Technical Conference, USENIX ATC 21, USENIX Association, 2021, pp. 413–426.

[63] G. Hinton, L. Deng, D. Yu, G.E. Dahl, A.-r. Mohamed, N. Jaitly, A. Senior, V. Vanhoucke, P. Nguyen, T.N. Sainath, B. Kingsbury, Deep neural networks for acoustic modeling in speech recognition: The shared views of four research groups, IEEE Signal Process. Mag. 29 (6) (2012) 82–97, http://dx.doi.org/10.1109/MSP.2012.2205597.

[64] M. Dundar, Q. Kou, B. Zhang, Y. He, B. Rajwa, Simplicity of kmeans versus deepness of deep learning: A case of unsupervised feature learning with limited data, in: 2015 IEEE 14th International Conference on Machine Learning and Applications, ICMLA, 2015, pp. 883–888, http://dx.doi.org/10.1109/ICMLA.2015.78.

[65] R. Li, S. Wang, F. Zhu, J. Huang, Adaptive graph convolutional neural networks, in: Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence and Thirtieth Innovative Applications of Artificial Intelligence Conference and Eighth AAAI Symposium on Educational Advances in Artificial Intelligence, in: AAAI'18/IAAI'18/EAAI'18, AAAI Press, 2018.

[66] Z. He, P. Chen, X. Li, Y. Wang, G. Yu, C. Chen, X. Li, Z. Zheng, A spatiotemporal deep learning approach for unsupervised anomaly detection in cloud systems, IEEE Trans. Neural Netw. Learn. Syst. (2020) 1–15, http://dx.doi.org/10.1109/TNNLS.2020.3027736.

[67] B. Perozzi, R. Al-Rfou, S. Skiena, DeepWalk: Online learning of social representations, in: Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, KDD '14, Association for Computing Machinery, New York, NY, USA, 2014, pp. 701–710, http://dx.doi.org/10.1145/2623330.2623732.

[68] J. Tang, M. Qu, M. Wang, M. Zhang, J. Yan, Q. Mei, LINE: Large-scale information network embedding, in: Proceedings of the 24th International Conference on World Wide Web, WWW '15, International World Wide Web Conferences Steering Committee, Republic and Canton of Geneva, CHE, 2015, pp. 1067–1077, http://dx.doi.org/10.1145/2736277.2741093.

[69] A. Grover, J. Leskovec, Node2vec: Scalable feature learning for networks, in: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, KDD '16, Association for Computing Machinery, New York, NY, USA, 2016, pp. 855–864, http://dx.doi.org/10.1145/2939672.2939754.

[70] T.N. Kipf, M. Welling, Semi-supervised classification with graph convolutional networks, 2017, arXiv:1609.02907.

[71] Heartbeat (computing), https://en.wikipedia.org/wiki/Heartbeat_(computing).

[72] H. Ren, B. Xu, Y. Wang, C. Yi, C. Huang, X. Kou, T. Xing, M. Yang, J. Tong, Q. Zhang, Time-series anomaly detection service at microsoft, in: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, KDD '19, Association for Computing Machinery, New York, NY, USA, 2019, pp. 3009–3017, http://dx.doi.org/10.1145/3292500.3330680.

[73] X. Hou, L. Zhang, Saliency detection: A spectral residual approach, in: 2007 IEEE Conference on Computer Vision and Pattern Recognition, 2007, pp. 1–8, http://dx.doi.org/10.1109/CVPR.2007.383267.

[74] K. Hundman, V. Constantinou, C. Laporte, I. Colwell, T. Soderstrom, Detecting spacecraft anomalies using LSTMs and nonparametric dynamic thresholding, in: Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, KDD '18, Association for Computing Machinery,

New York, NY, USA, 2018, pp. 387–395, http://dx.doi.org/10.1145/3219819.3219845.
[75] Ryu, https://ryu-sdn.org.
[76] Hipster-Shop: Sample cloud-native application with 10 microservices, https://github.com/GoogleCloudPlatform/microservices-demo.
[77] Alibaba ecommerce microservice benchmark, https://github.com/alibaba/eCommerceSearchBench.
[78] Chaosblade, https://github.com/chaosblade-io/chaosblade.
[79] S. Knight, H. Nguyen, N. Falkner, R. Bowden, M. Roughan, The internet topology zoo, IEEE J. Sel. Areas Commun. 29 (9) (2011) 1765–1775, http://dx.doi.org/10.1109/JSAC.2011.111002.
[80] R. Ben Basat, S. Ramanathan, Y. Li, G. Antichi, M. Yu, M. Mitzenmacher, PINT: Probabilistic in-band network telemetry, in: Proceedings of the Annual Conference of the ACM Special Interest Group on Data Communication on the Applications, Technologies, Architectures, and Protocols for Computer Communication, SIGCOMM '20, Association for Computing Machinery, New York, NY, USA, 2020, pp. 662–680, http://dx.doi.org/10.1145/3387514.3405894.
[81] Z. Liu, A. Manousis, G. Vorsanger, V. Sekar, V. Braverman, One sketch to rule them all: Rethinking network flow monitoring with UnivMon, in: Proceedings of the 2016 ACM SIGCOMM Conference, SIGCOMM '16, Association for Computing Machinery, New York, NY, USA, 2016, pp. 101–114, http://dx.doi.org/10.1145/2934872.2934906.
[82] A. Samanta, J. Tang, Dyme: Dynamic microservice scheduling in edge computing enabled IoT, IEEE Internet Things J. 7 (7) (2020) 6164–6174.
[83] P. Jin, X. Fei, Q. Zhang, F. Liu, B. Li, Latency-aware VNF chain deployment with efficient resource reuse at network edge, in: IEEE INFOCOM 2020 - IEEE Conference on Computer Communications, IEEE Press, 2020, pp. 267–276, http://dx.doi.org/10.1109/INFOCOM41043.2020.9155345.
[84] tc, https://man7.org/linux/man-pages/man8/tc.8.html.
[85] N. Tatbul, T.J. Lee, S. Zdonik, M. Alam, J. Gottschlich, Precision and recall for time series, in: Proceedings of the 32nd International Conference on Neural Information Processing Systems, NIPS '18, Curran Associates Inc., Red Hook, NY, USA, 2018, pp. 1924–1934.
[86] A.H. Yaacob, I.K. Tan, S.F. Chien, H.K. Tan, ARIMA based network anomaly detection, in: 2010 Second International Conference on Communication Software and Networks, 2010, pp. 205–209, http://dx.doi.org/10.1109/ICCSN.2010.55.
[87] M.-L. Shyu, S.-C. Chen, K. Sarinnapakorn, L. Chang, A novel anomaly detection scheme based on principal component classifier, in: Proceedings of International Conference on Data Mining, 2003.
[88] M.M. Breunig, H.-P. Kriegel, R.T. Ng, J. Sander, LOF: Identifying density-based local outliers, SIGMOD Rec. 29 (2) (2000) 93–104.



Hongyang Chen received his BE degree from Sun Yat-sen University, China, in 2020 and is currently pursuing the Ph.D. degree in the School of Computer Science and Engineering, Sun Yat-sen University. His current research areas focus on distributed system, cloud computing, chaos engineering and software defined networking.



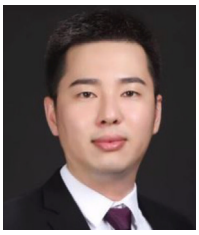
Pengfei Chen is currently an associated professor in the School of Computer Science and Engineering of Sun Yat-sen University. Meanwhile, he is a Ph.D. advisor. Dr. Chen graduated from the department of computer science of Xi'an Jiaotong University with a Ph.D. degree in 2016. Now, he is interested in distributed systems, AIOps, cloud computing, Microservice and BlockChain. Especially, he has strong skills in cloud computing. So far, Dr. Chen has published more than 70 papers in some international conferences including ACM ASE/ICSE, IEEE INFOCOM, WWW, ACM/IEEE CCGRID, ISSRE, IEEE ICWS, IEEE DSN and journals including IEEE TDSC, IEEE TNNLS, IEEE TR, IEEE TSC, IEEE TETC, IEEE TCC. He serves as of program committee member of multiple conferences and reviewers of some internal journals such as IEEE Transactions on Cybernetics, Information Science, and Neurocomputing.



Benran Wang received his BE degree from Sun Yat-sen University, China, and is currently pursuing the Master degree in the School of Computer Science and Engineering, Sun Yat-sen University. His current research areas focus on software defined networking and programmable networking.



Xian Yu is currently a research and development expert of cloud computing technology in Sangfor Technologies Inc, Shenzhen, China. He received the Ph.D. degree in Institute of Computing Technology, Chinese Academy of Sciences, Beijing, China, in 2020. His research interests include network architecture, AIOps, cloud computing, intelligent network technology, and green network. His research works have been published in FPL, FCCM, Globecom, ICC, ICCBR, JCRD (in chinese) and so on.



Xiaofan Chen is currently a cloud architect in Sangfor Technologies Inc, Shenzhen, China. He received the B.S. and Ph.D. degrees in Sun Yat-Sen University, Guangzhou, China. His research interests include software-defined networks, AIOps and cloud computing.



Dandan Ma graduated with a Master's degree in Radio Physics from Jilin University and currently work in the Intelligent Team of the Network Product Innovation Center of China United Network Communications Co., Ltd. Guangdong Province Company, responsible for operator intelligence related work.



Zibin Zheng is currently a Professor and the Deputy Dean with the School of Software Engineering, Sun Yat-sen University, Guangzhou, China. He obtained the Ph.D. degree in Computer Science and Engineering at the Chinese University of Hong Kong. He authored or coauthored more than 200 international journal and conference papers, including one ESI hot paper and six ESI highly cited papers. His research interests include blockchain, software engineering, and services computing. He is a Fellow of the IEEE and the IET. He was the recipient of several awards, including the Top 50 Influential Papers in Blockchain of 2018, the ACM SIGSOFT Distinguished Paper Award at ICSE2010, the Best Student Paper Award at ICWS2010.